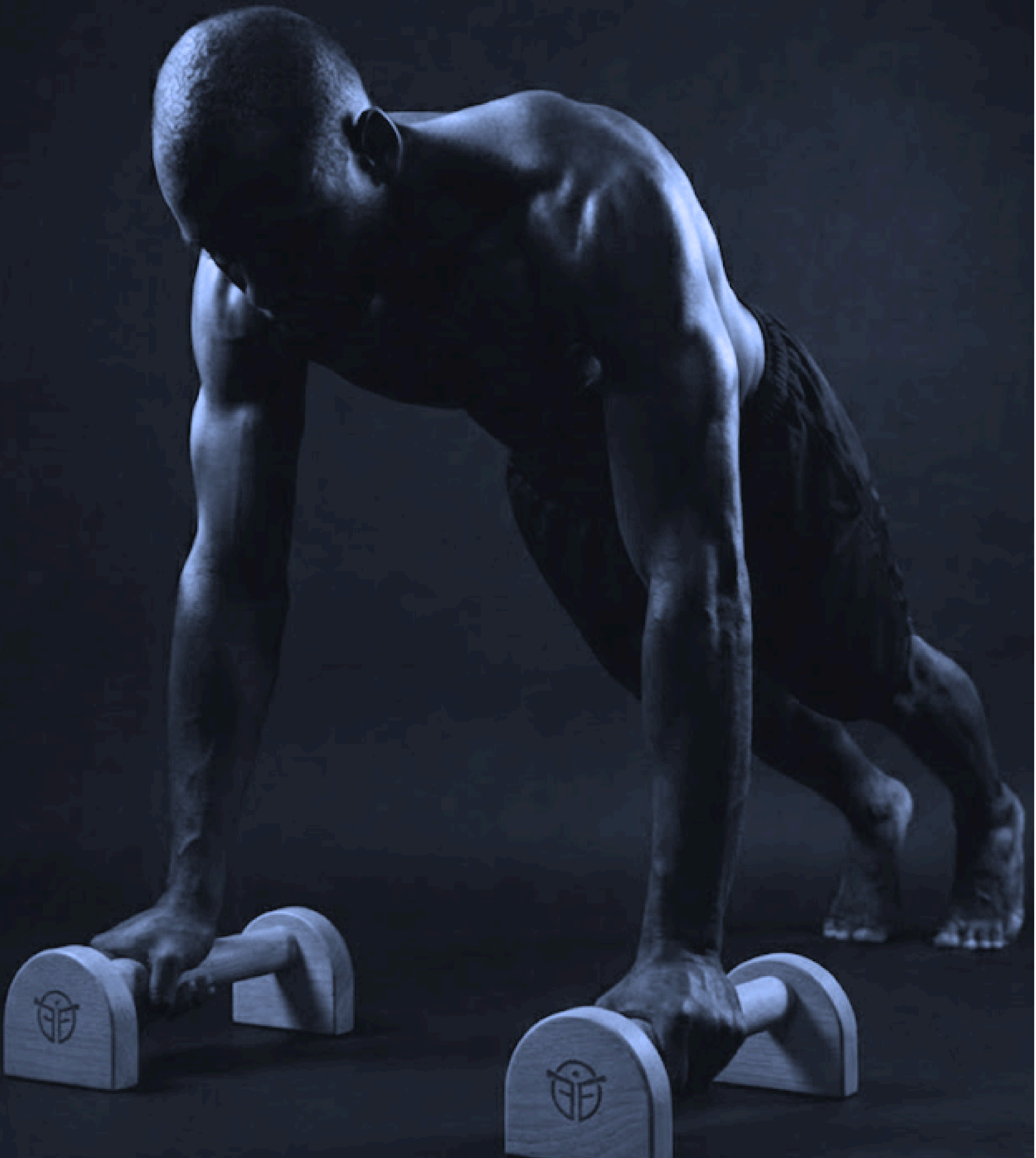


Incredibly **K**een **E**ngineers **A**ssociation

PushApp



Indice

Introduzione	3
Strumenti di Lavoro	3
1. Funzionalità	4
1.1 Autenticazione e Onboarding	4
1.2 Statistiche	4
1.3 Gestione Allenamenti	4
1.4 Allenamento	5
1.5 Storico	5
2. Architettura	6
2.1 Guida al Diagramma Architetturale	6
2.1.1 UI Layer	6
2.1.2 Presentation Layer	6
2.1.3 Data Layer	6
2.1.4 External Systems	6
2.2 Struttura dei Package	8
2.2.1 UI	8
2.2.2 ViewModels	8
2.2.3 Adapters	8
2.2.4 Repositories	8
2.2.5 Database	8
2.2.6 API	8
2.2.7 Models	8
2.2.8 Utils	8
3. Design	10
3.1 Layouts	10
3.2 Color Palette e Tipografia	11
4. Conclusioni	18
4.1 Risultati Chiave e Punti di Forza	18
4.2 Sfide Incontrate e Lezioni Apprese	19
4.3 Sviluppi Futuri e Prospettive	19

Introduzione

PushApp è un'applicazione mobile progettata per ridefinire l'approccio di allenamento, trasformando il tradizionale diario di allenamento cartaceo in uno strumento analitico avanzato. L'app nasce per rispondere all'esigenza di atleti e frequentatori di palestre che necessitano di tracciare con precisione i propri progressi, basandosi sul principio del sovraccarico progressivo.

Attraverso un'interfaccia intuitiva, PushApp permette all'utente di creare schede di allenamento personalizzate, registrare le sessioni in tempo reale e monitorare metriche chiave come il volume di allenamento e il massimo peso. Grazie a PushApp la pianificazione incontra l'analisi: grazie all'integrazione di grafici dettagliati e storici delle performance, l'utente può visualizzare i propri miglioramenti nel tempo, trasformando i dati grezzi in insight azionabili per l'ottimizzazione delle performance atletiche.

Strumenti di Lavoro

Per lo sviluppo di PushApp sono stati utilizzati diversi strumenti e tecnologie, orchestrati per garantire un'architettura robusta, scalabile e manutenibile.

- **Android Studio** è stato l'ambiente di sviluppo utilizzato per la realizzazione dell'app. Il linguaggio di programmazione scelto è Java, sfruttato per la sua stabilità e per la gestione esplicita della concorrenza.
- **GitHub** ha rappresentato la piattaforma per il versionamento del codice. La gestione del progetto ha seguito un flusso di lavoro strutturato per garantire l'integrità del codice sorgente:
 - Il branch *main* contiene le versioni stabili e pronte per il deployment.
 - I branch *develop* funge da ambiente di sviluppo condiviso, dove le modifiche o le correzioni vengono caricate prima che queste siano integrate nella versione finale.
 - Le fasi intermedie di sviluppo si sono svolte principalmente su branch creati *ad hoc*, creati specificamente per l'implementazione di una determinata funzionalità.
- **Firebase**, la suite di servizi backend di Google, ricopre un ruolo cruciale nell'ecosistema PushApp:
 - **Firestore**: Utilizzato come database NoSQL remoto per il backup e la sincronizzazione dei dati.
 - **Authentication**: Gestisce il sistema di login e registrazione, assicurando che ogni User acceda esclusivamente ai propri dati sensibili.
- **Room Database** è la libreria di persistenza fondamentale per l'architettura "Offline-First" dell'app. Esso gestisce il database relazionale permettendo all'utente di allenarsi e salvare dati anche in assenza di connessione.

- **Retrofit & Ninja API:** per arricchire l'esperienza utente, è stata integrata la Ninja API tramite la libreria Retrofit. Questo servizio esterno permette di popolare il database con un vasto catalogo di esercizi, filtrabili per gruppo muscolare e difficoltà.
- Infine, per la visualizzazione dei dati, è stata integrata la libreria **MPAndroidChart**. Questa permette di renderizzare grafici interattivi che mostrano l'andamento del volume e dei carichi nel tempo.

1. Funzionalità

1.1 Autenticazione e Onboarding

L'accesso all'ecosistema PushApp è progettato per essere sicuro e personalizzato fin dal primo utilizzo.

- **Login e Registrazione:** L'utente può accedere tramite credenziali classiche (e-mail e password). È prevista una procedura di recupero password in caso di smarrimento delle credenziali.
- **Onboarding:** Al primo accesso, l'utente viene guidato attraverso un percorso di configurazione per definire i parametri biometrici essenziali.

1.2 Statistiche

All'avvio, all'utente viene mostrata la parte più analitica dell'applicazione. Questa schermata funge da hub centrale, unendo la motivazione immediata all'analisi di lungo periodo:

- **KPI in Evidenza:** Visualizzazione immediata di Streak (giorni consecutivi), Workouts mensili e Volume totale.
- **Calendario Attività:** Una vista a calendario permette di visualizzare la frequenza di allenamento mensile, evidenziando i giorni di attività.
- **Grafici di Performance:** Integrazione di MPAndroidChart per visualizzare l'andamento del Volume Totale e del Progresso di Peso per ogni singolo esercizio, permettendo di valutare il sovraccarico progressivo a colpo d'occhio.

1.3 Gestione Allenamenti

Questa sezione rappresenta il cuore della pianificazione, permettendo all'utente di strutturare i propri allenamenti con precisione chirurgica.

- **Struttura Gerarchica:** L'utente può creare molteplici programmi di allenamento. Ogni programma contiene diverse Routine (giorni di allenamento) personalizzabili.
- **Editor:** Attraverso l'EditRoutineFragment, è possibile costruire la scheda tecnica. L'app si interfaccia con la Ninja API per fornire un database vastissimo di esercizi, filtrabili per gruppo muscolare e livello di difficoltà, facilitando la scoperta e l'inserimento di nuovi movimenti.

1.4 Allenamento

In questa schermata il tuo smartphone si trasforma in un assistente di sala digitale durante l'esecuzione degli esercizi.

- **Live Tracking:** Un'interfaccia ottimizzata per il data-entry rapido permette di registrare serie, ripetizioni e carico in tempo reale. Il sistema supporta la modifica dinamica dei tempi di recupero tra le serie.
- **Timer Intelligente:** Un timer globale monitora la durata della sessione, mentre un Rest Timer automatico gestisce i tempi di recupero tra i set.
- **Mini-Player Persistente:** Una funzionalità distintiva di PushApp è il Mini-Player. Se l'utente naviga in altre sezioni dell'app durante un allenamento, una barra inferiore permette di monitorare il timer e tornare rapidamente alla sessione attiva o scartarla, garantendo un'esperienza multitasking fluida.

1.5 Storico

Questo modulo funge da registro cronologico dettagliato.

- **Log Sessioni:** Elenco completo di tutti gli allenamenti completati, recuperati tramite HistoryRepository.
- **Dettaglio Serie:** Ogni voce è espandibile per mostrare i dettagli tecnici (kg x reps) di ogni serie eseguita.
- **Ricerca e Gestione:** Funzionalità di ricerca testuale per trovare rapidamente vecchie sessioni e opzioni per la loro eliminazione.

2. Architettura

L'architettura di PushApp è stata progettata seguendo i principi del pattern **MVVM** (Model-View-ViewModel), supportato dal **Repository Pattern**. Questa scelta garantisce una netta separazione delle responsabilità (Separation of Concerns), facilitando la testabilità, la manutenibilità e la gestione del ciclo di vita dei componenti Android.

Il sistema adotta un approccio "Offline-First": il database locale (Room) funge da Single Source of Truth per l'interfaccia utente, mentre i dati vengono sincronizzati in background con il cloud (Firebase) e arricchiti da servizi esterni (Ninja API). La gestione delle dipendenze è orchestrata tramite un Service Locator Pattern, che fornisce le istanze necessarie ai vari componenti senza accoppiamenti rigidi.

2.1 Guida al Diagramma Architeturale

2.1.1 UI Layer

Questo blocco rappresenta l'interfaccia con cui l'utente interagisce. Osserva i dati, ma non contiene logica di business.

- **Activities**
- **Fragments**
- **Adapters**

2.1.2 Presentation Layer

Questo blocco gestisce lo stato della UI e reagisce agli eventi utente. Comunica con il Data Layer e espone i dati tramite *LiveData*.

- **ViewModels**

2.1.3 Data Layer

Questo blocco astrae la provenienza dei dati. I ViewModel non sanno se i dati arrivano dal DB o dal Cloud

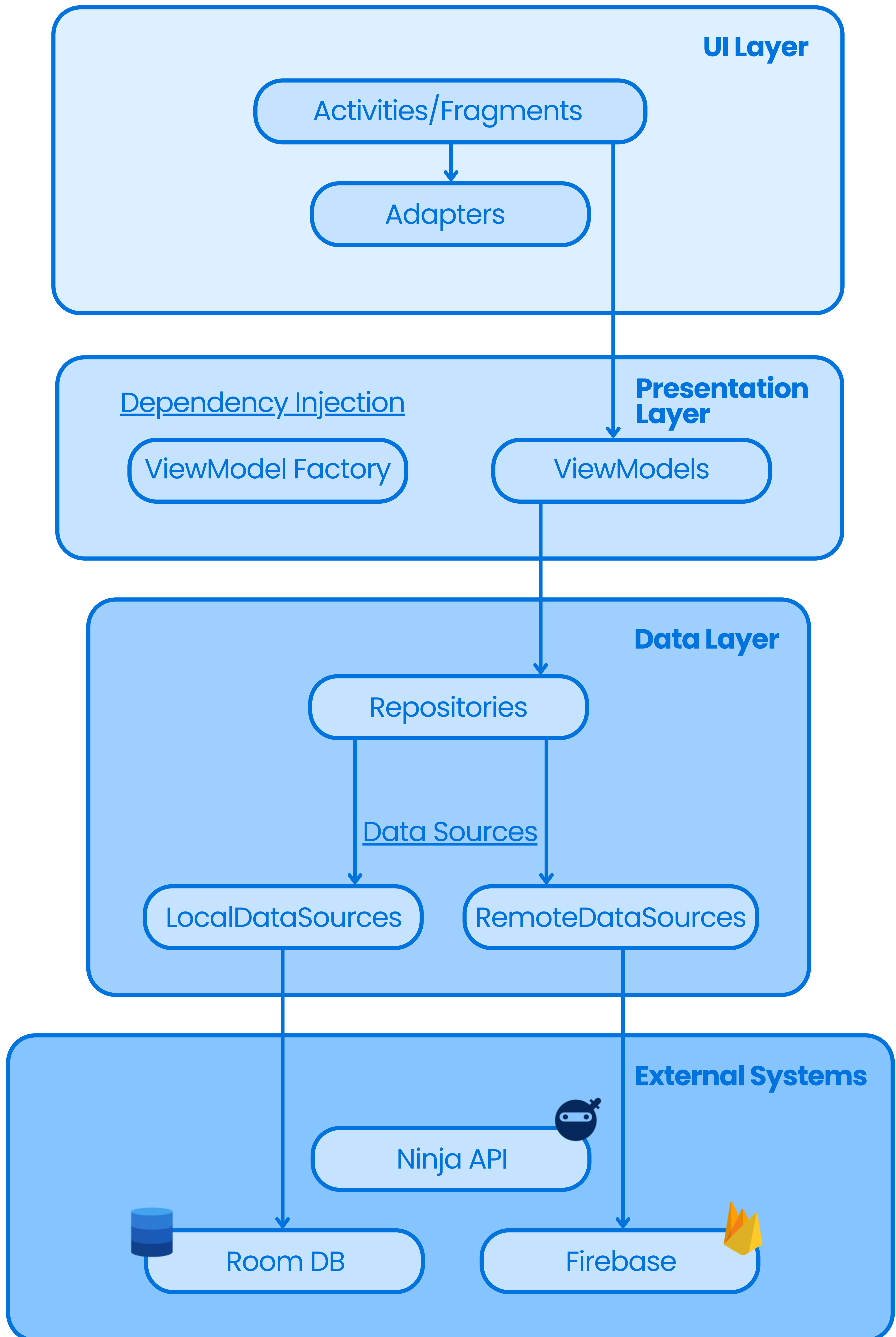
- **Repositories**
- **DataSources**

2.1.4 External Systems

Le tecnologie e i servizi con cui l'app comunica.

- **Local DB:** Room Database
- **Cloud Backend:** Firebase Firestore, Firebase Auth
- **External API:** Ninja API (via Retrofit)

Rappresentazione visiva della Struttura dell'Architettura



2.2 Struttura dei Package

Il codice sorgente è organizzato semanticamente per riflettere i layer architetturali.

2.2.1 UI

Contiene tutti i componenti relativi all'interfaccia utente. È suddiviso in sotto-package per funzionalità (login, main) e contiene i Fragment che osservano i ViewModel e gli Adapter per le RecyclerView.

2.2.2 ViewModels

Ospita le classi che estendono ViewModel. Questi componenti sopravvivono ai cambiamenti di configurazione e fungono da ponte tra la UI e i dati.

2.2.3 Adapters

Questo package ospita le classi Adapter, componenti cruciali per il popolamento delle RecyclerView. Essi fungono da ponte tra i modelli di dati e i componenti visivi, gestendo il binding delle viste e le interazioni dell'utente.

2.2.4 Repositories

Il cuore della gestione dei dati. Qui risiede la logica di sincronizzazione: il Repository decide quando leggere dal database locale e quando effettuare chiamate di rete. Include anche il ServiceLocator per l'iniezione delle dipendenze.

2.2.5 Database

Definisce la struttura del database locale utilizzando la libreria Room. Contiene le Entity, i DAO (Data Access Objects) e la configurazione del database.

2.2.6 API

Gestisce la comunicazione con le API REST di terze parti. Utilizza Retrofit per definire gli endpoint e convertire le risposte JSON in oggetti Java.

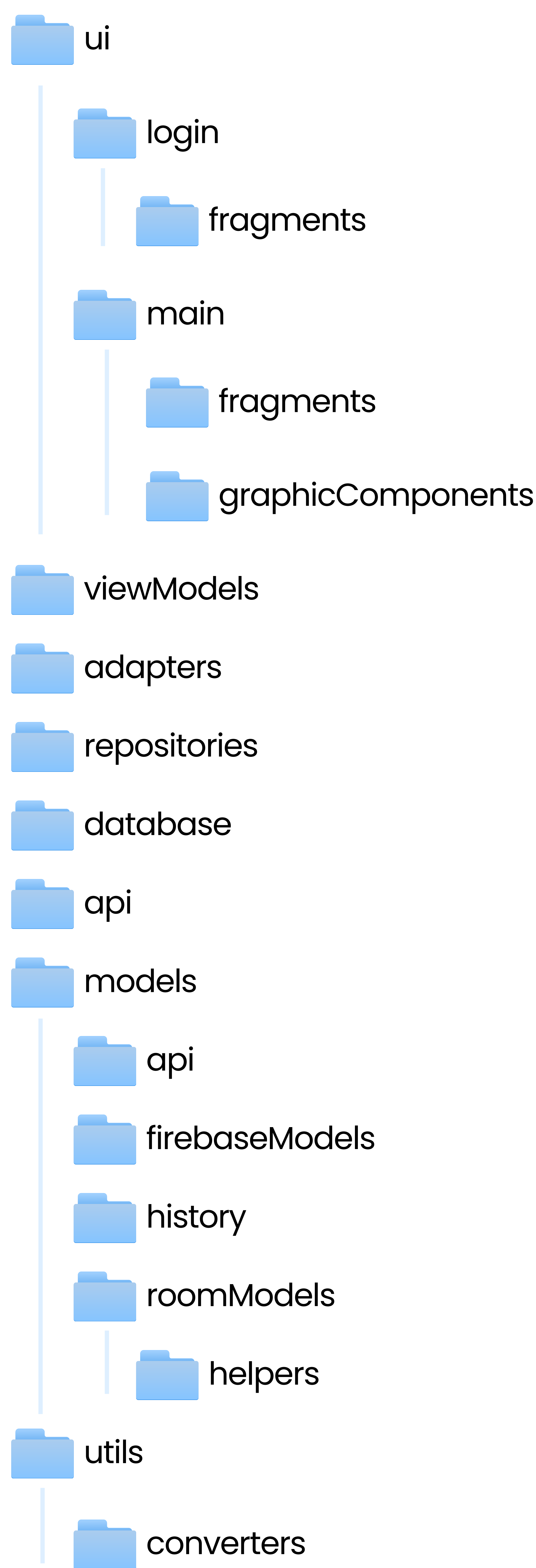
2.2.7 Models

Contiene le classi POJO (Plain Old Java Objects) e le Entity che rappresentano i dati dell'applicazione.

2.2.8 Utils

Raccoglie classi di utilità trasversali, helpers e converters.

Rappresentazione visiva della Struttura dei Package



3. Design

Il design di PushApp è stato concepito per rispondere a una sfida specifica: offrire un'interfaccia avanzata e ricca di dati che rimanga tuttavia immediata, semplice e priva di distrazioni durante l'attività fisica intensa. L'interfaccia grafica aderisce rigorosamente alle linee guida del **Material Design 3**, garantendo coerenza visiva, accessibilità e un'esperienza utente fluida su tutto l'ecosistema Android.

3.1 Layouts

La progettazione dei layout ha seguito un approccio function-first. Considerando che l'utente interagisce con l'app nei tempi di recupero o durante l'esecuzione degli esercizi, l'interfaccia privilegia la chiarezza e la rapidità di input.

- **Struttura a Card:** L'ecosistema visivo si basa sull'uso di MaterialCardView. Questo permette di raggruppare logicamente le informazioni in blocchi distinti e facilmente scansionabili dall'occhio umano.
- **Gerarchia Visiva:** Nei layout critici come il WorkoutFragment, la gerarchia è definita dalle dimensioni e dal posizionamento. Il Timer e i controlli di riproduzione sono sempre ancorati e visibili, mentre le liste di esercizi utilizzano Nested RecyclerViews per gestire la complessità dei dati senza appesantire la schermata.
- **Data Visualization:** Per la sezione statistiche, i layout sono stati ripuliti da ogni elemento superfluo. Grazie alla classe ChartHelper, i grafici sono renderizzati con uno stile minimalista: griglie rimosse, assi semplificati e focus totale sulla trendline per comunicare il progresso a colpo d'occhio.

3.2 Color Palette e Tipografia

La palette cromatica di PushApp è definita semanticamente nel file colors.xml per supportare nativamente sia il tema Chiaro che quello Scuro, essenziale per il risparmio energetico e il comfort visivo in ambienti con illuminazione artificiale come le palestre.

Per la tipografia è stato scelto il font di sistema Roboto, molto presente nell'ecosistema Android per la sua eccellente leggibilità.

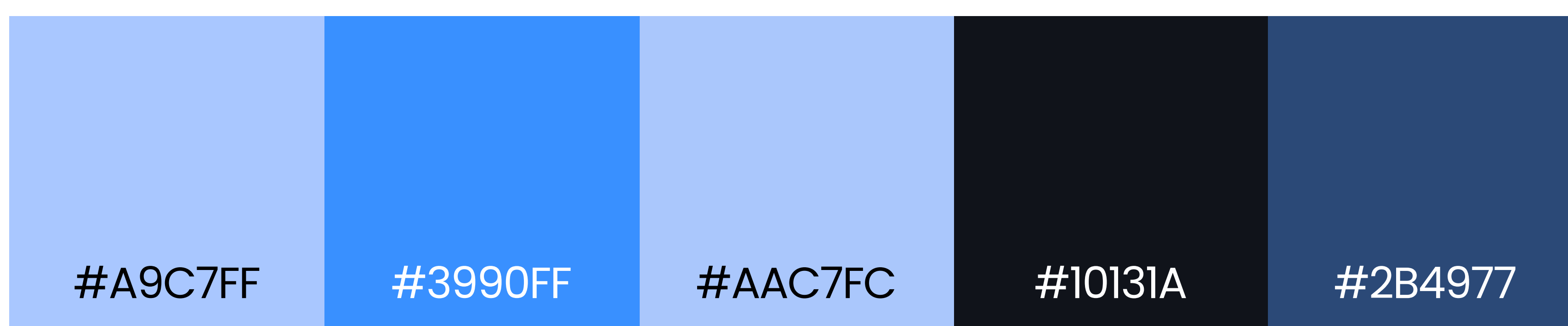
L'app sfrutta diverse pesature per creare contrasto:

- **Roboto Bold:** Utilizzato per i KPI numerici per renderli leggibili a colpo d'occhio.
- **Roboto Regular:** Utilizzato per i testi descrittivi e le istruzioni degli esercizi.
- **Roboto Medium:** Impiegato nei pulsanti e nelle tab di navigazione per garantire chiarezza nelle call-to-action.

Di seguito vengono mostrati i colori utilizzati per entrambi i temi:



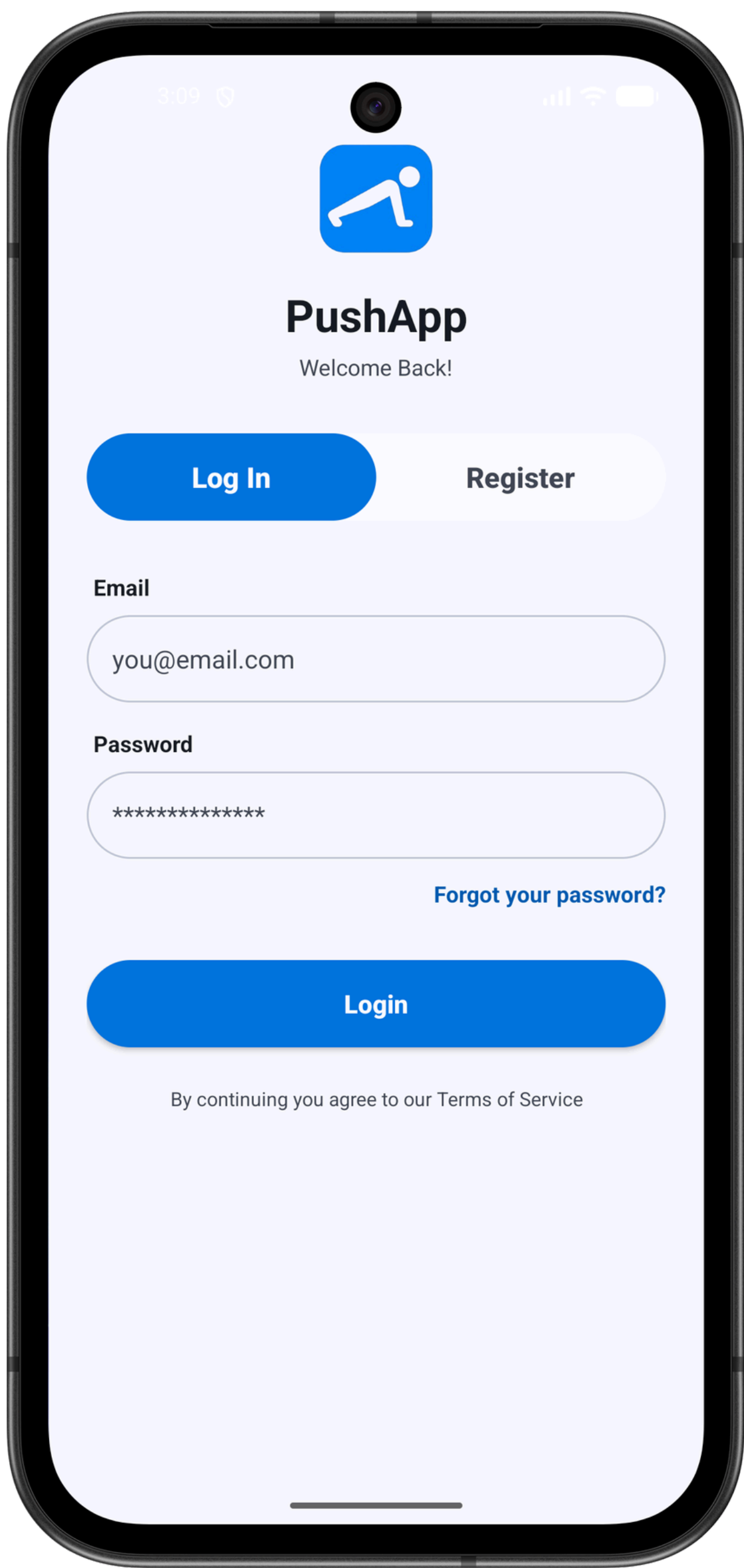
(a) Palette per il tema chiaro



(b) Palette per il tema scuro

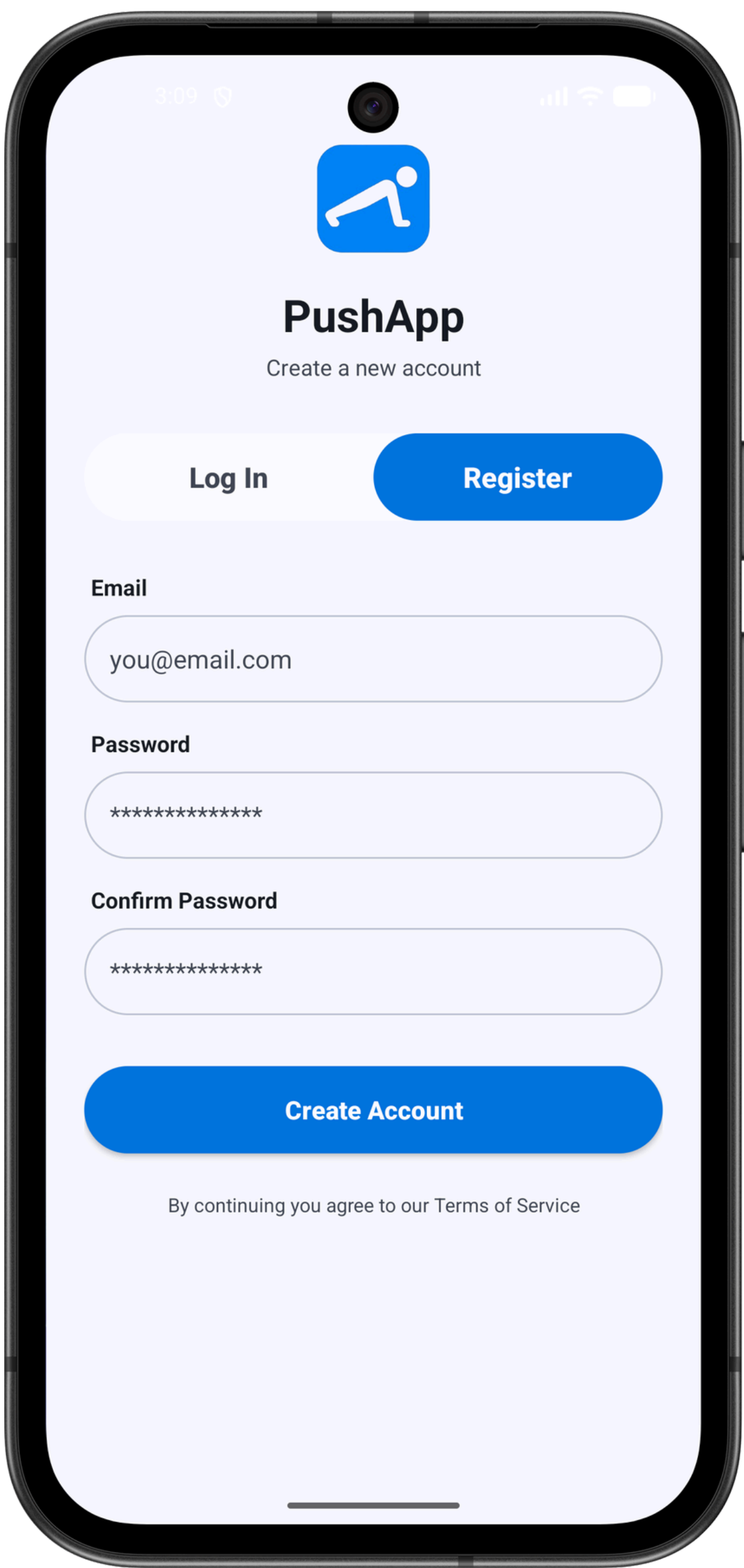
Schermate PushApp per il tema chiaro

Login



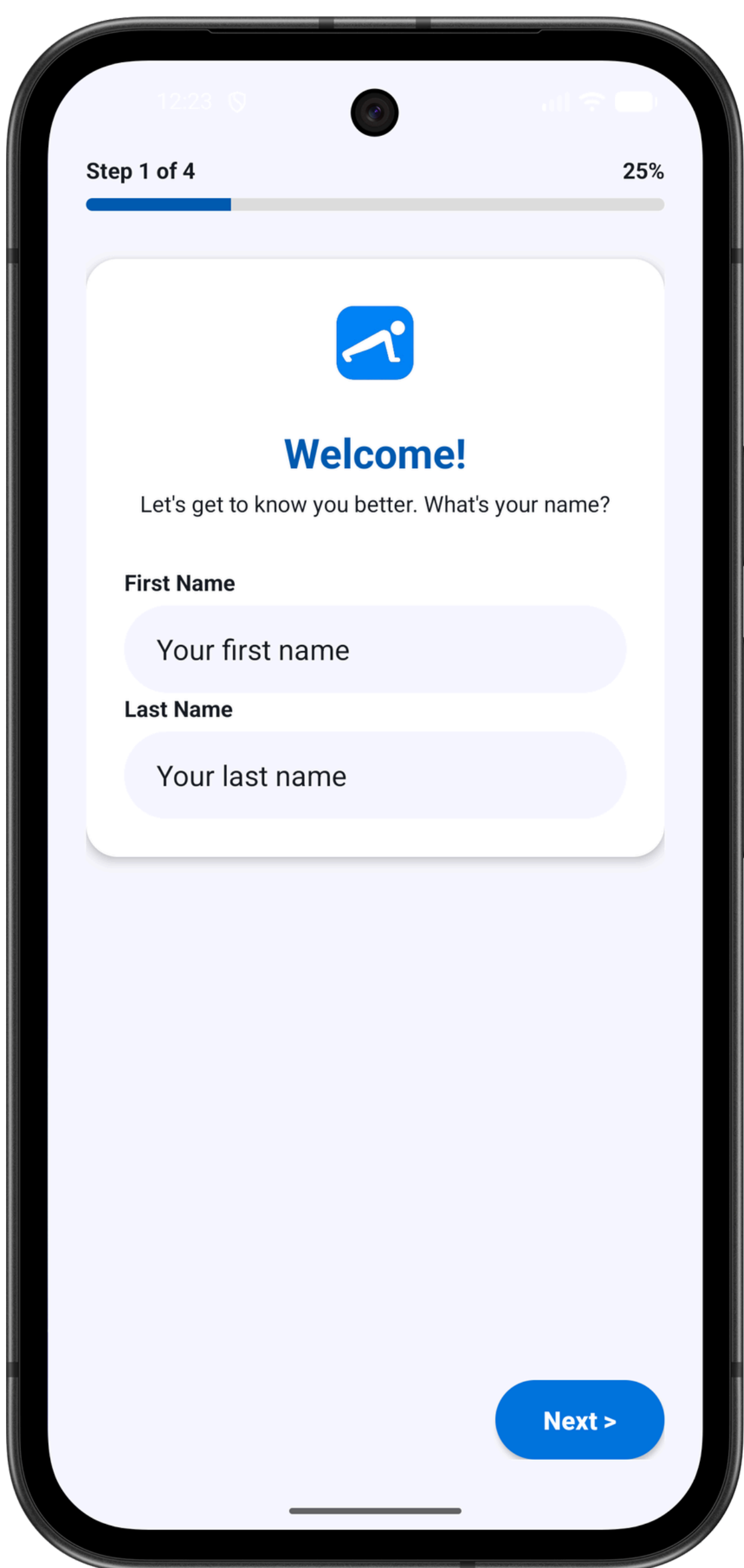
LoginFragment

Register



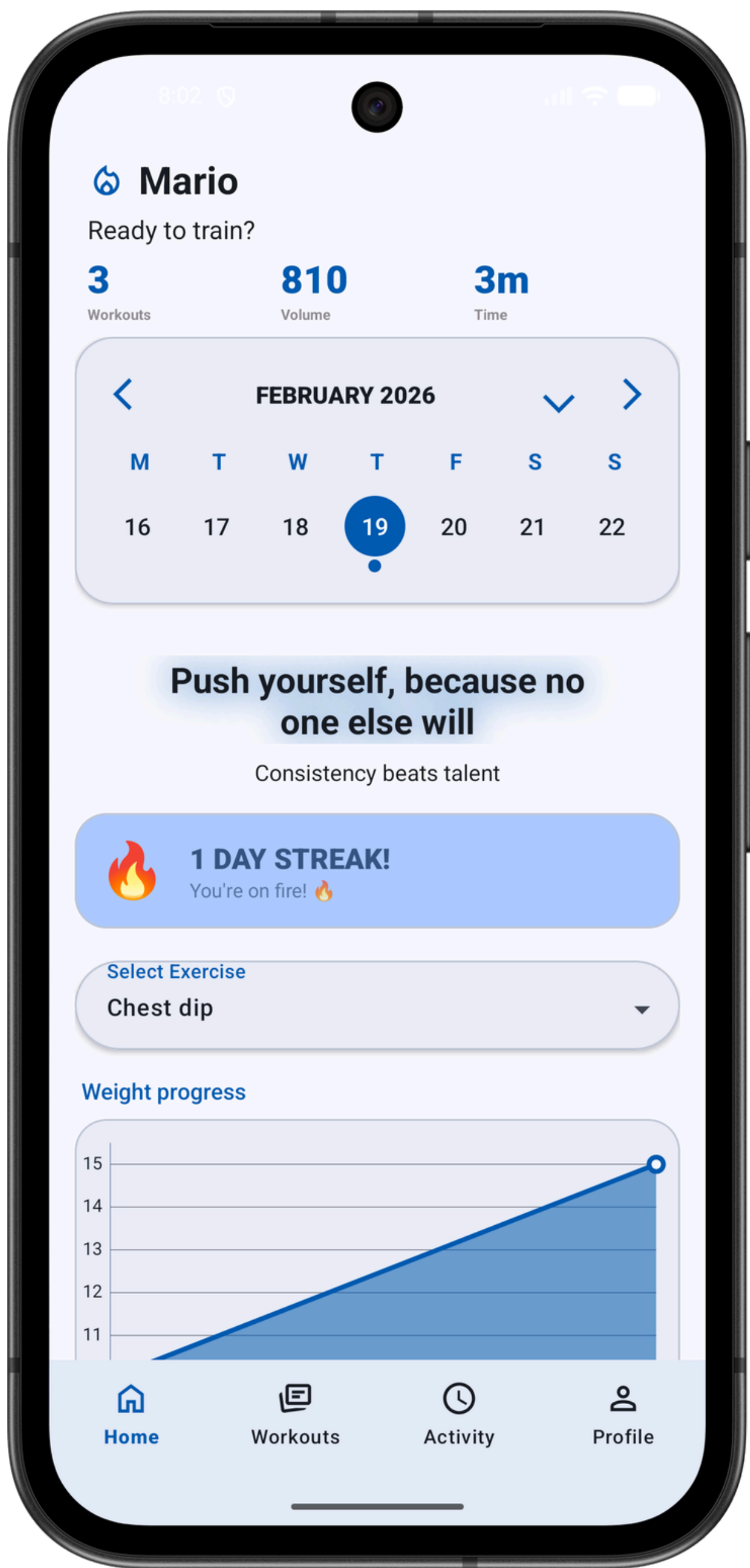
RegisterFragment

Onboarding

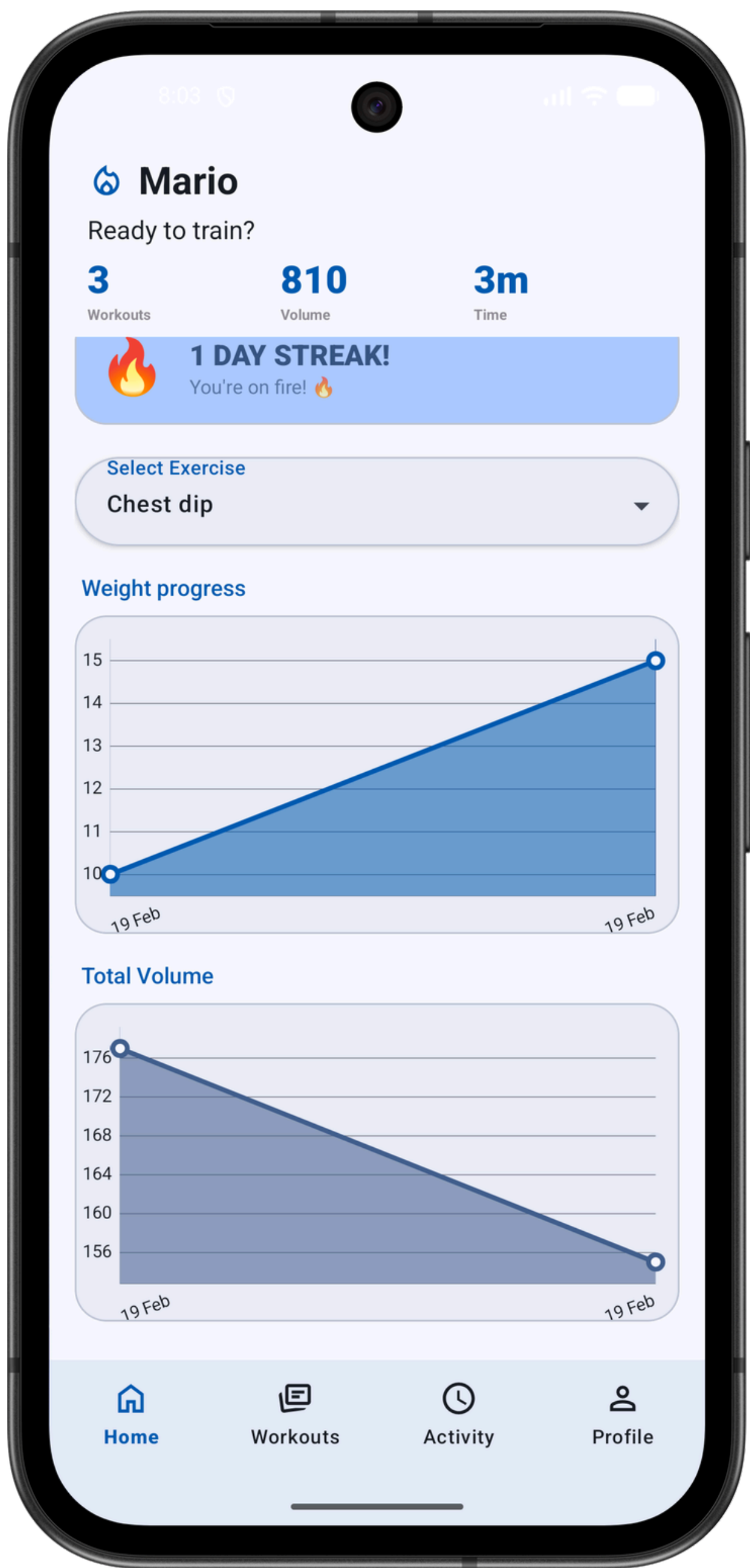


QuestionsActivity

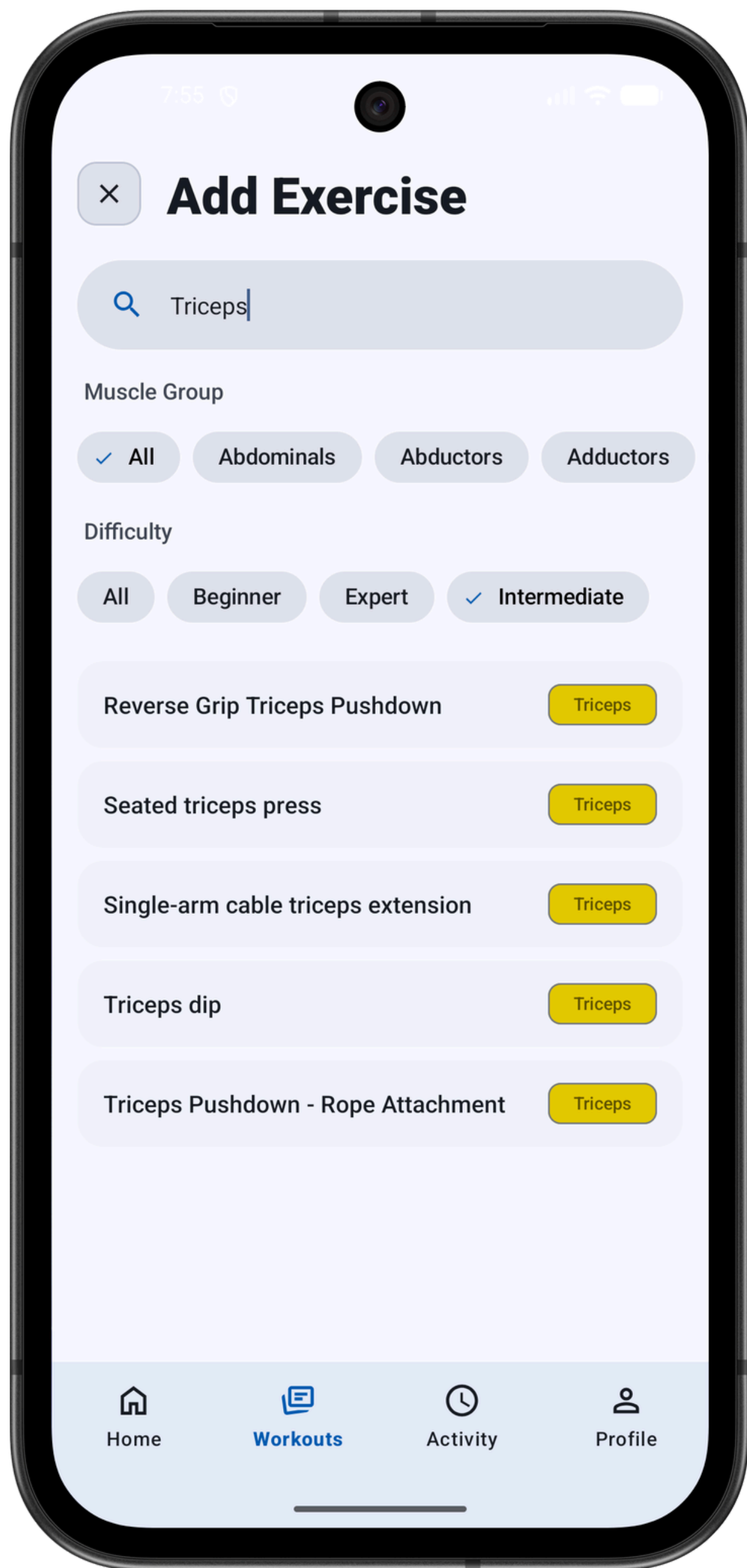
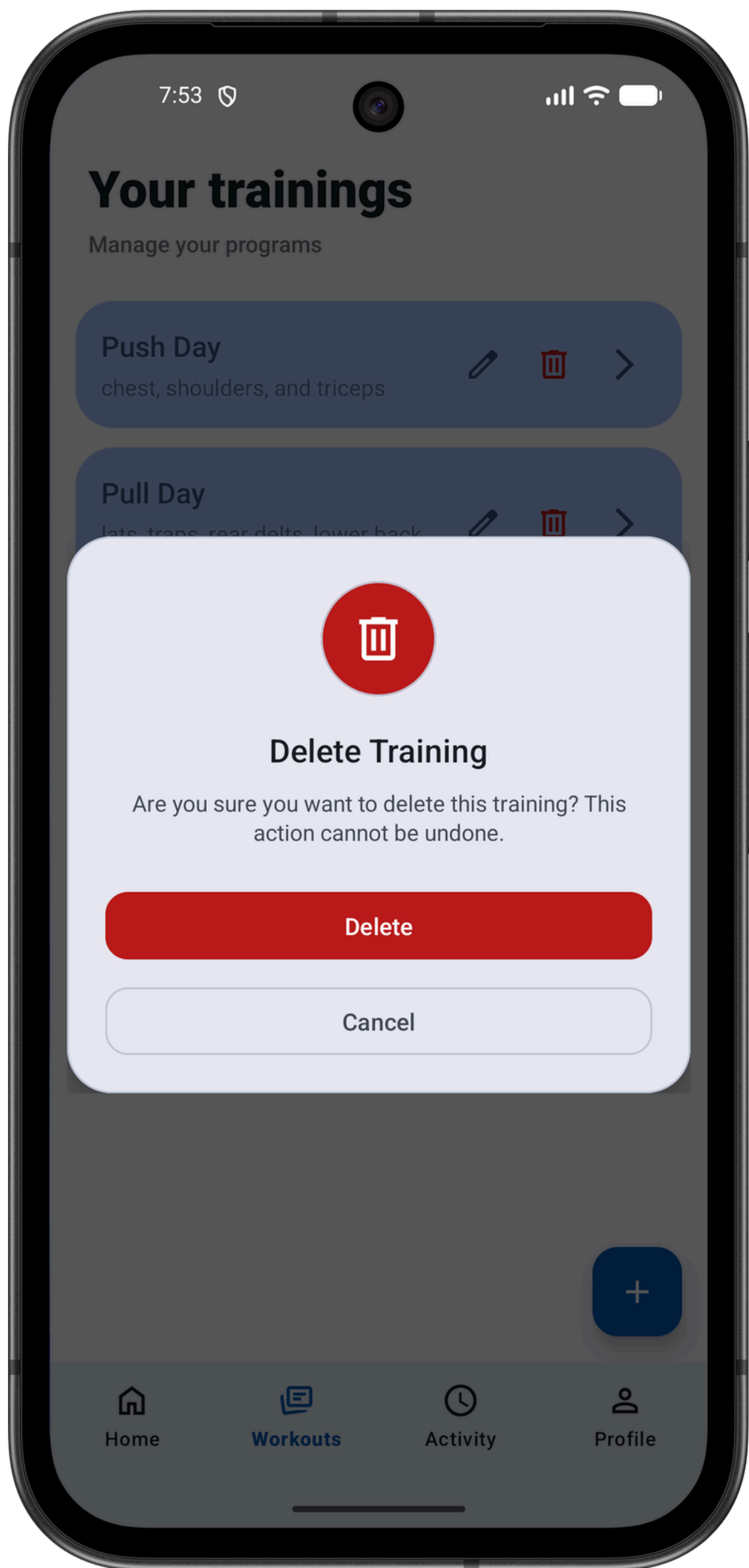
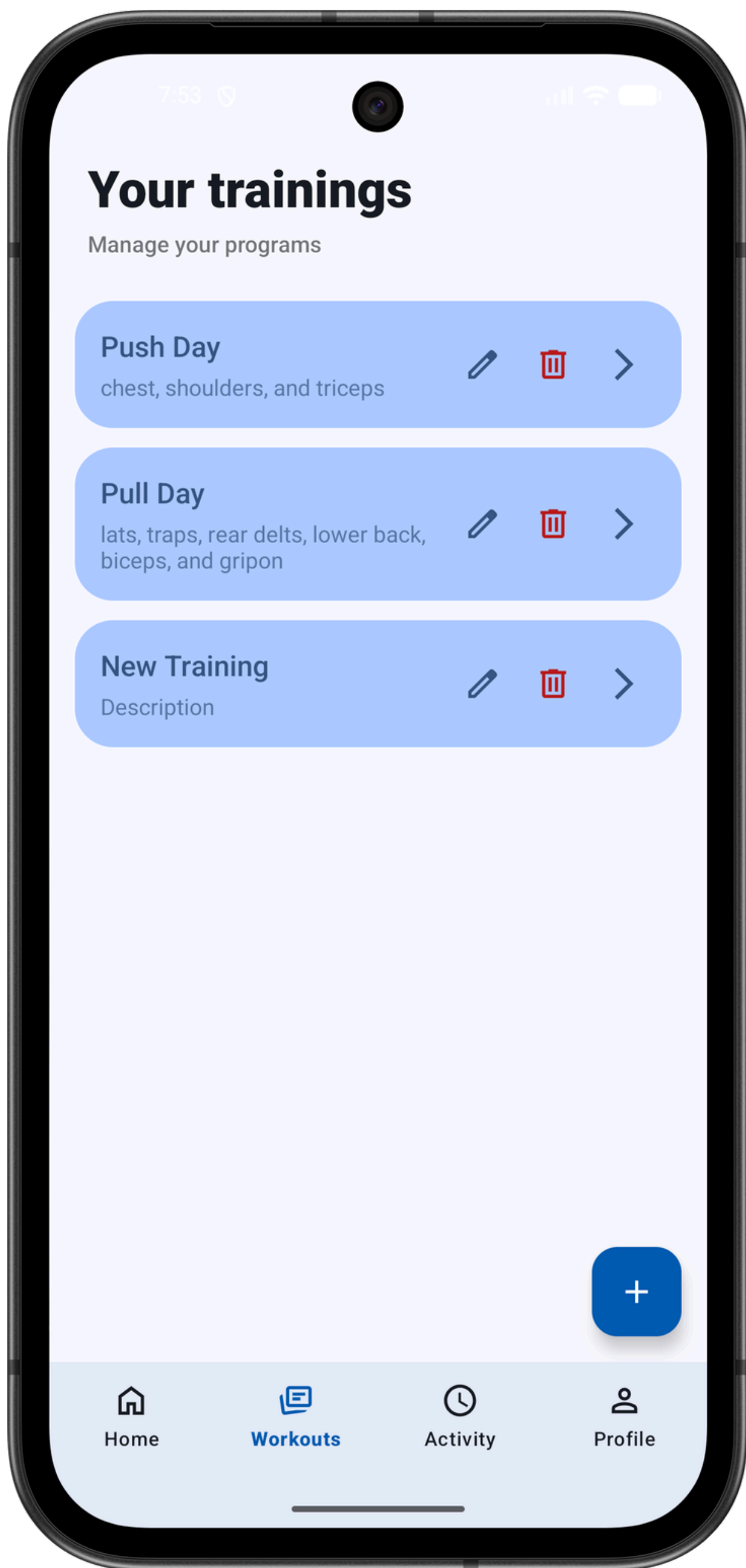
Home



HomeFragment

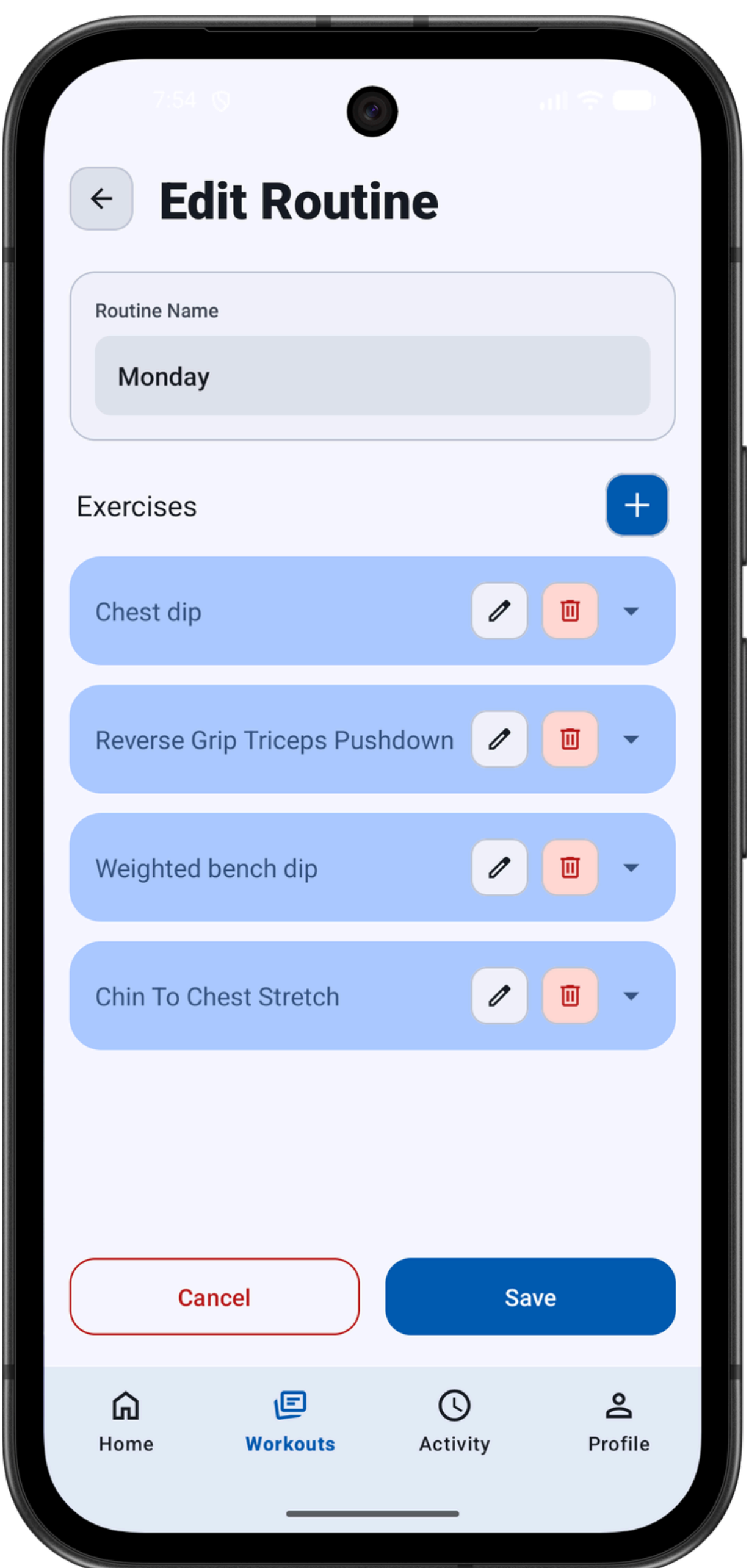
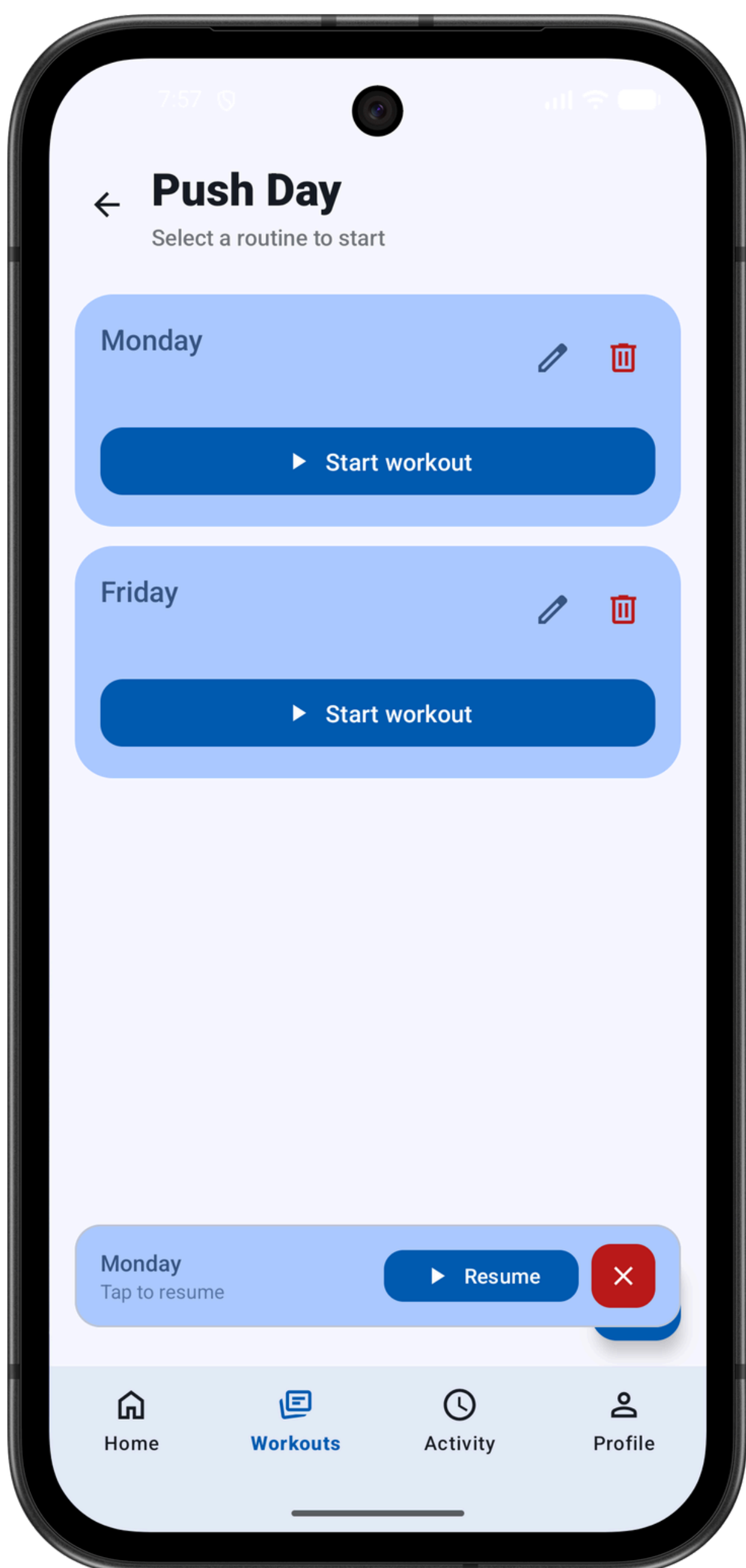


Trainings



TrainingsFragment

Routines

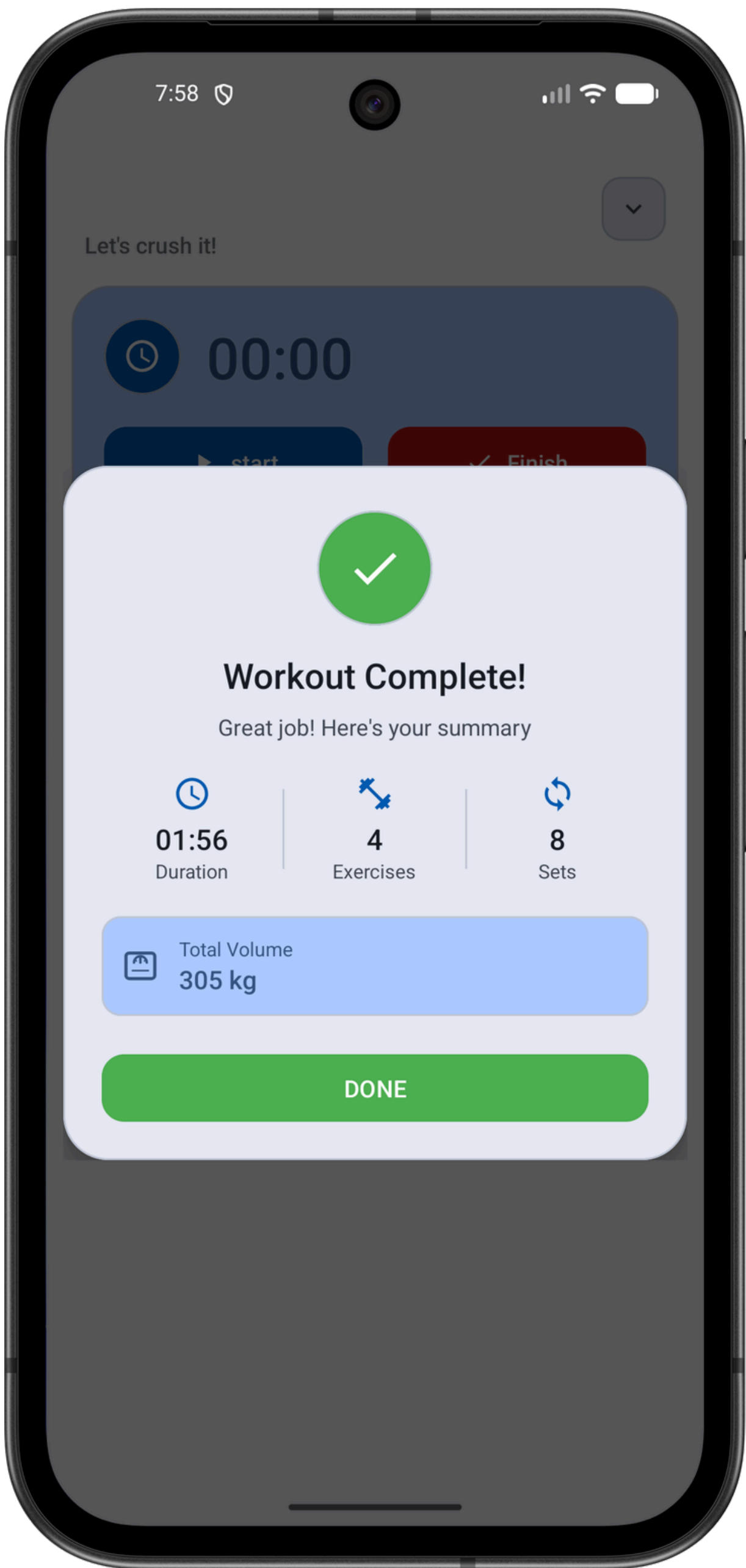
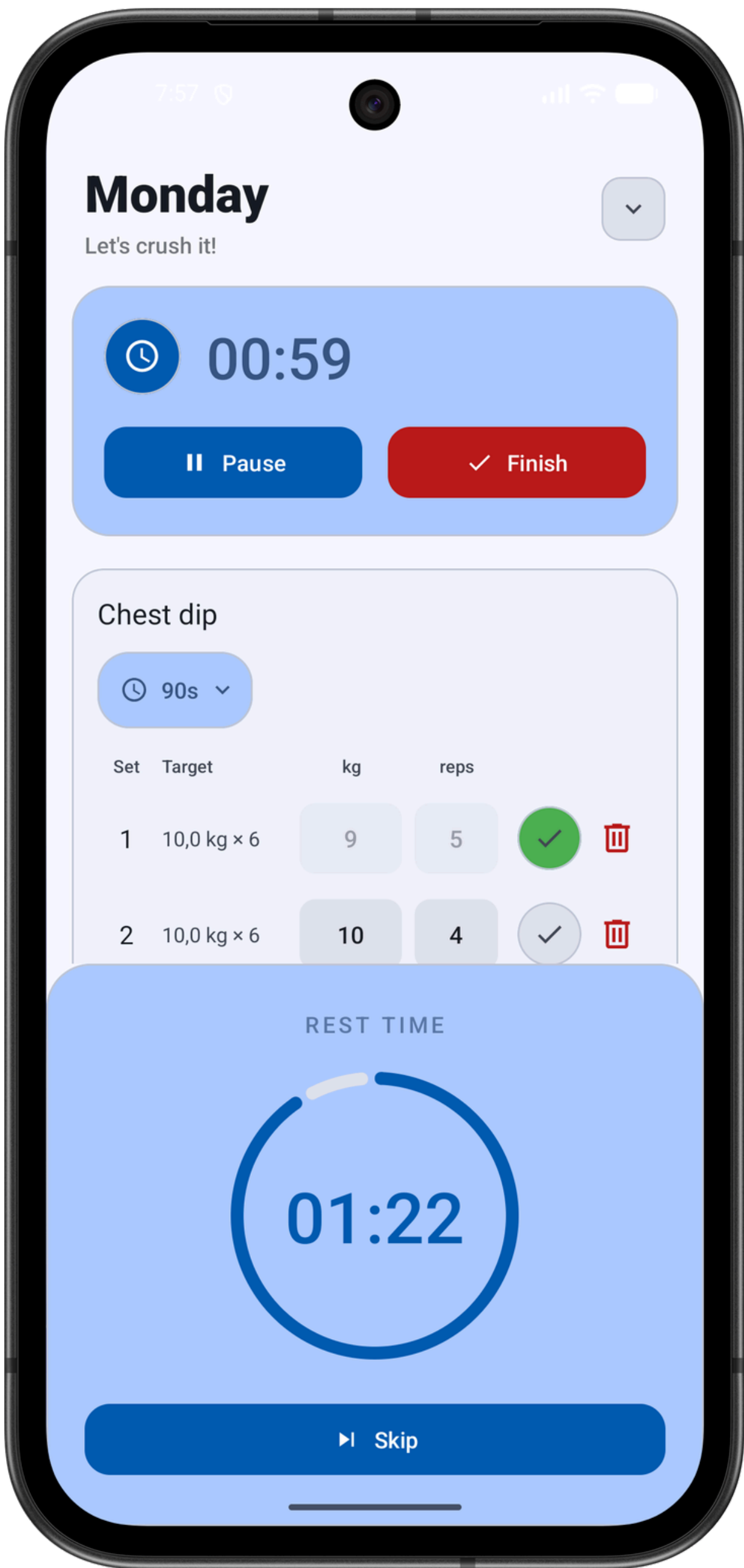


RoutineFragment

Edit Routine

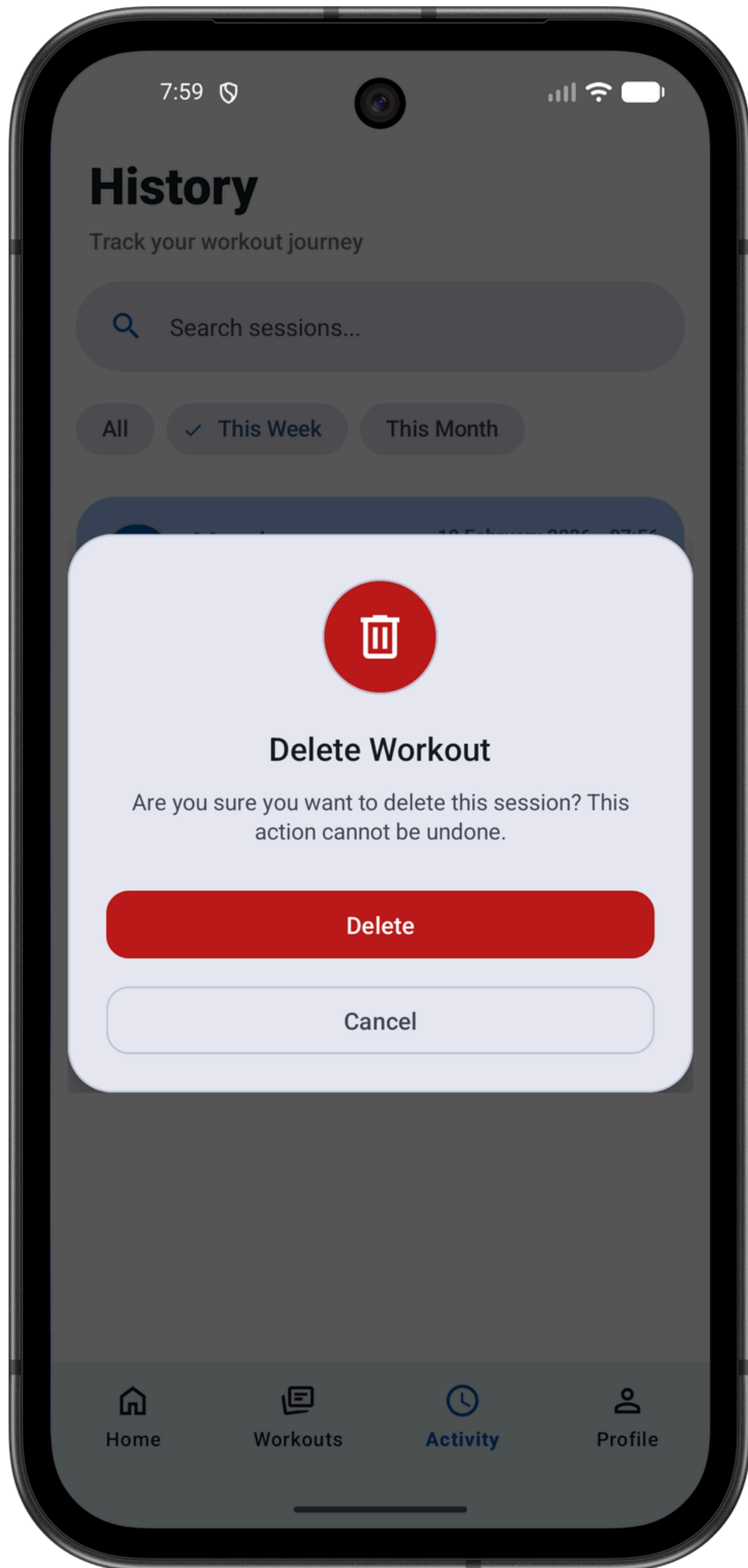
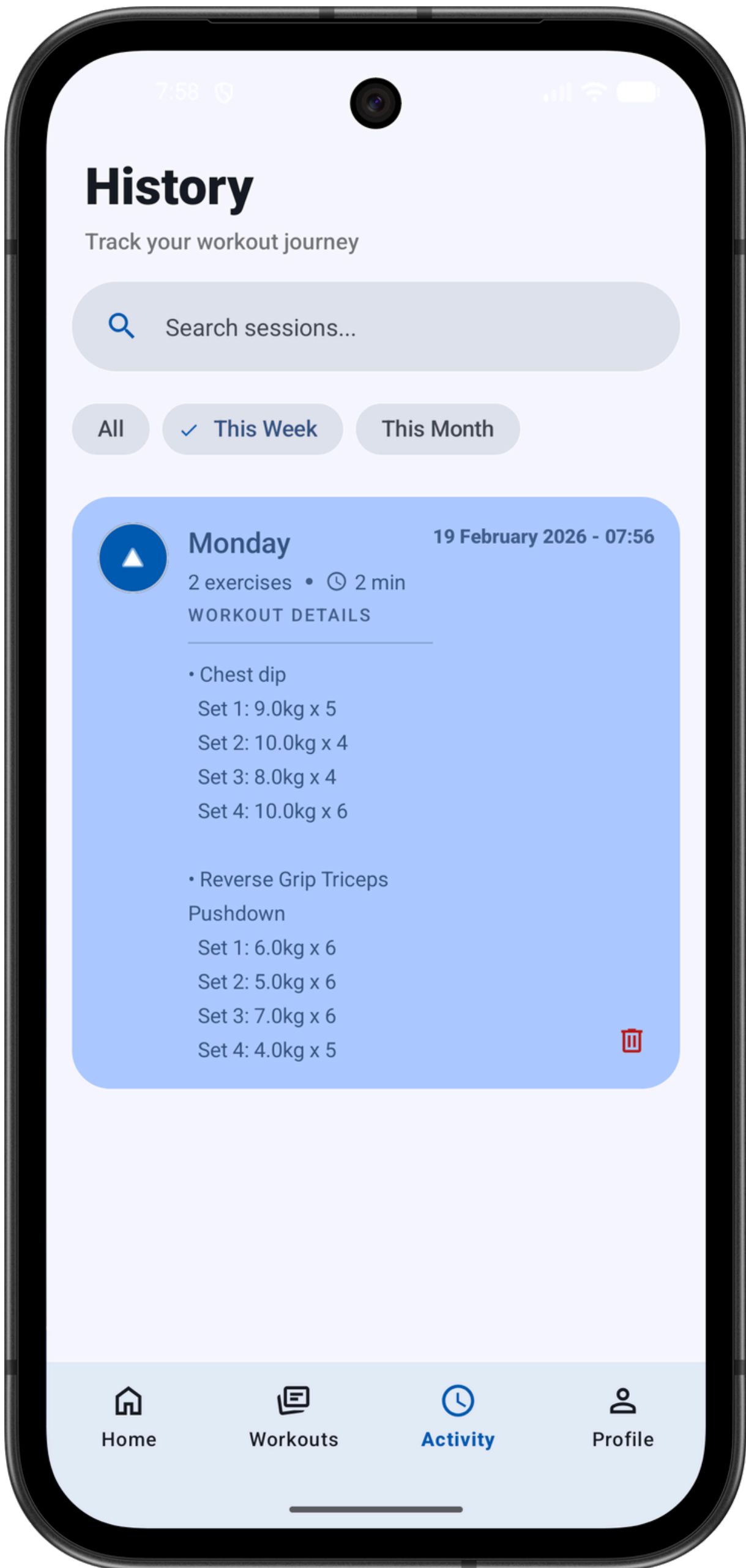
EditRoutineFragment

Workout



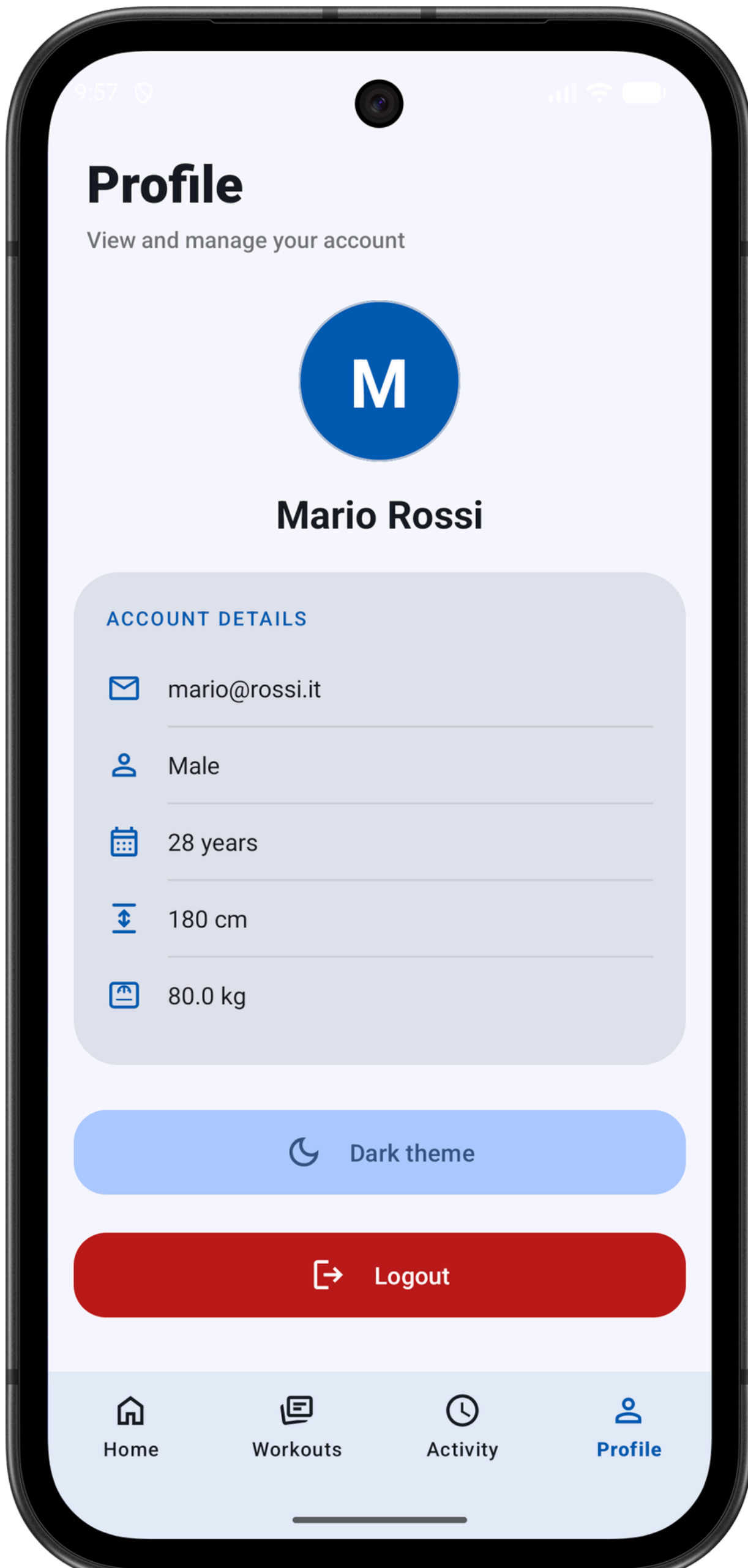
WorkoutFragment

History



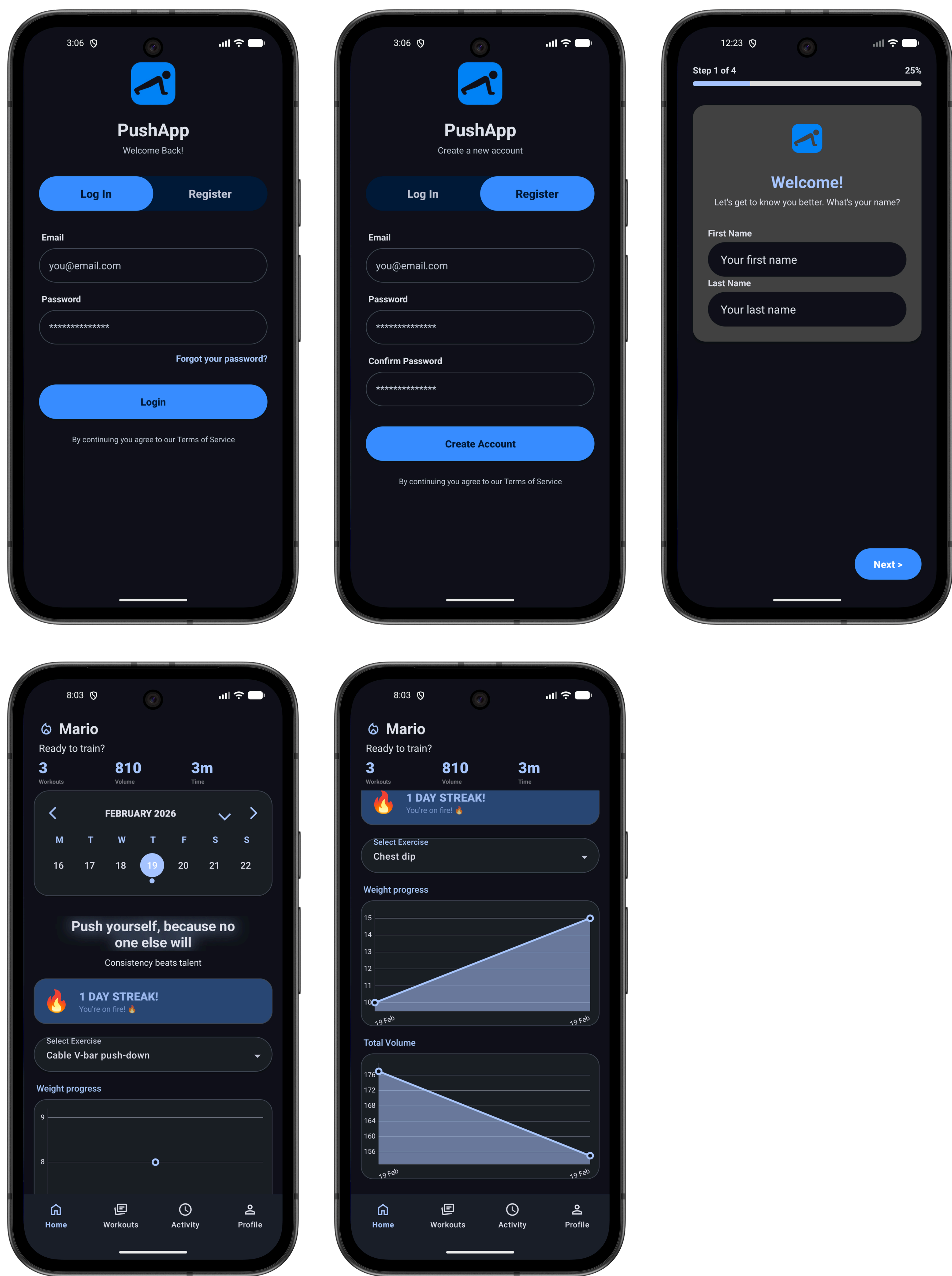
HistoryFragment

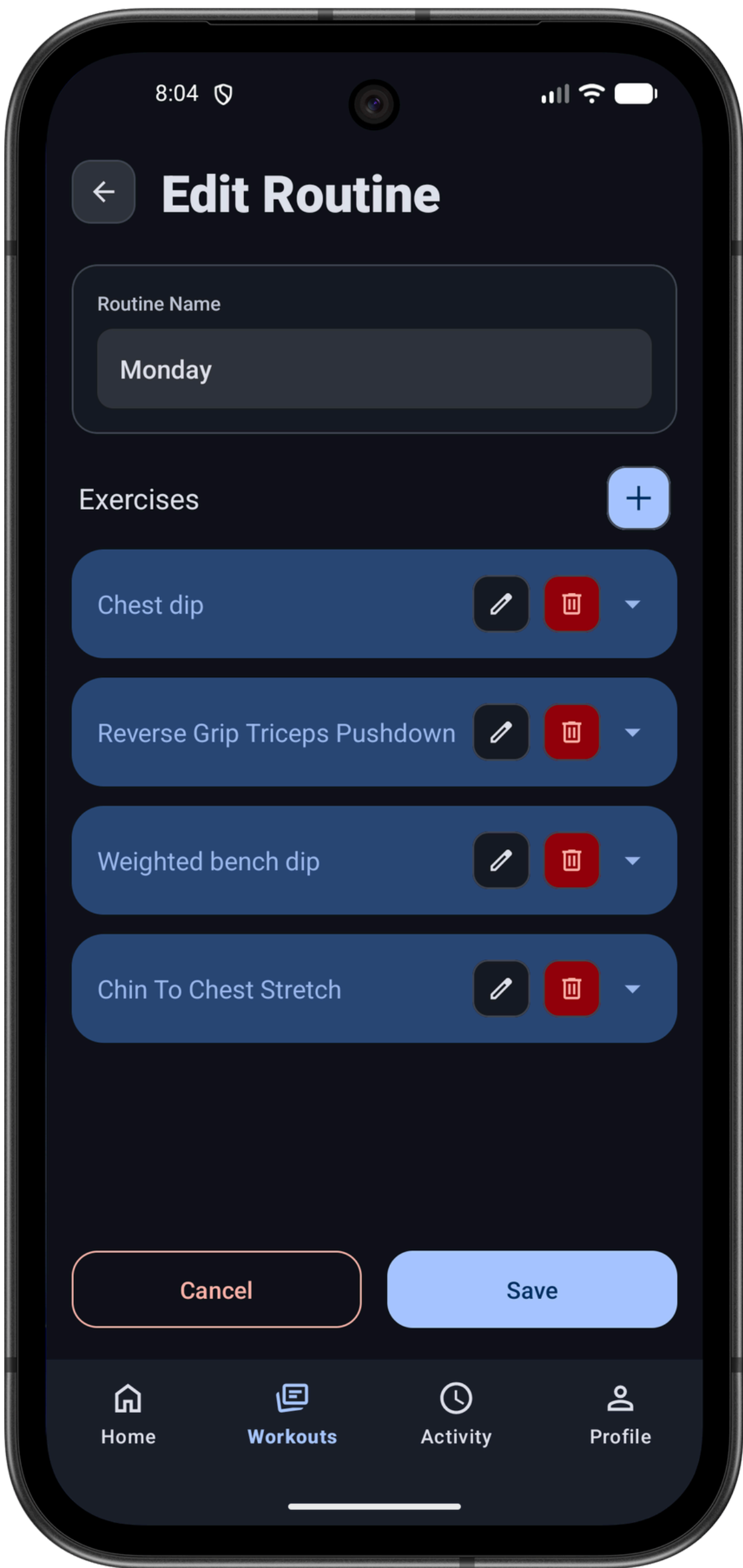
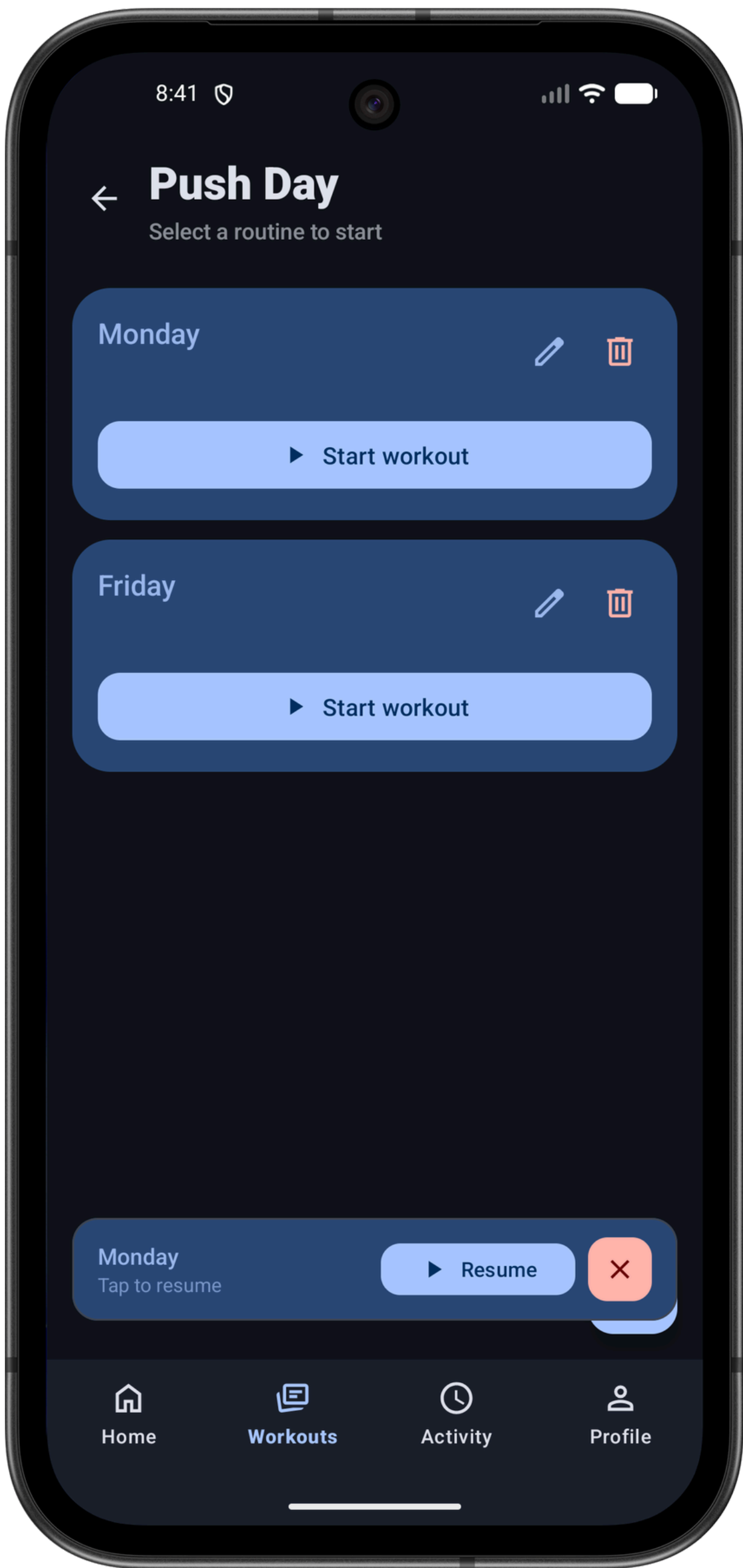
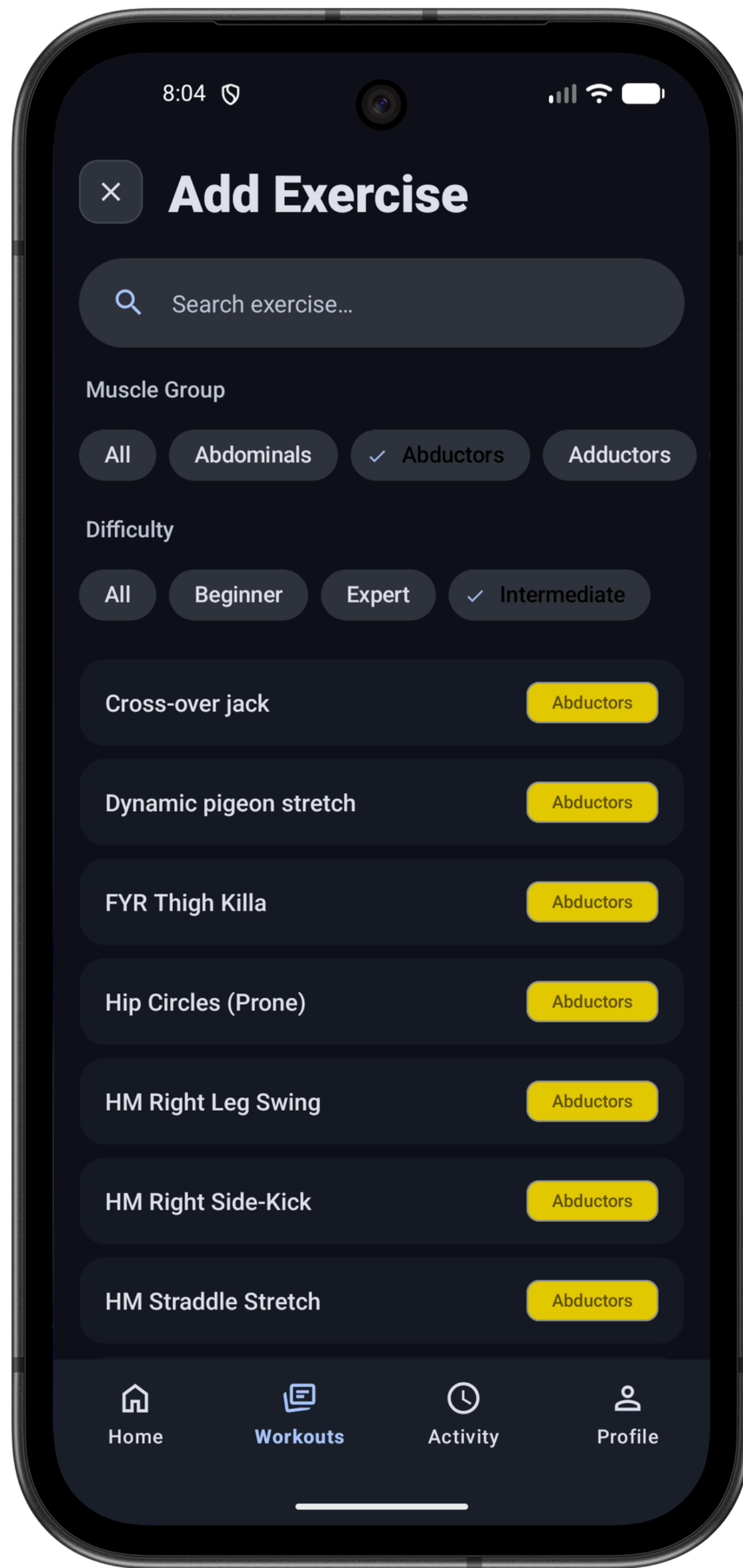
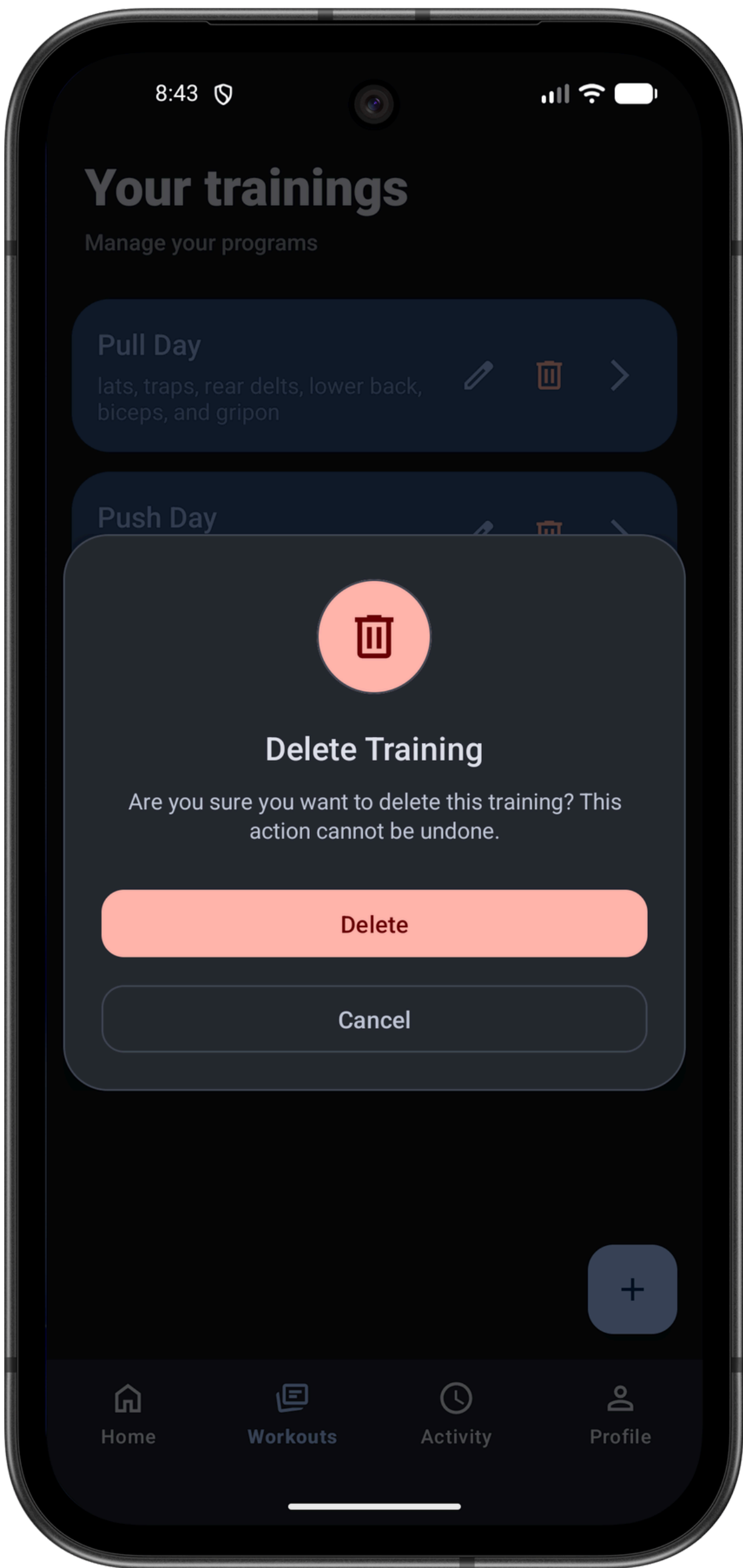
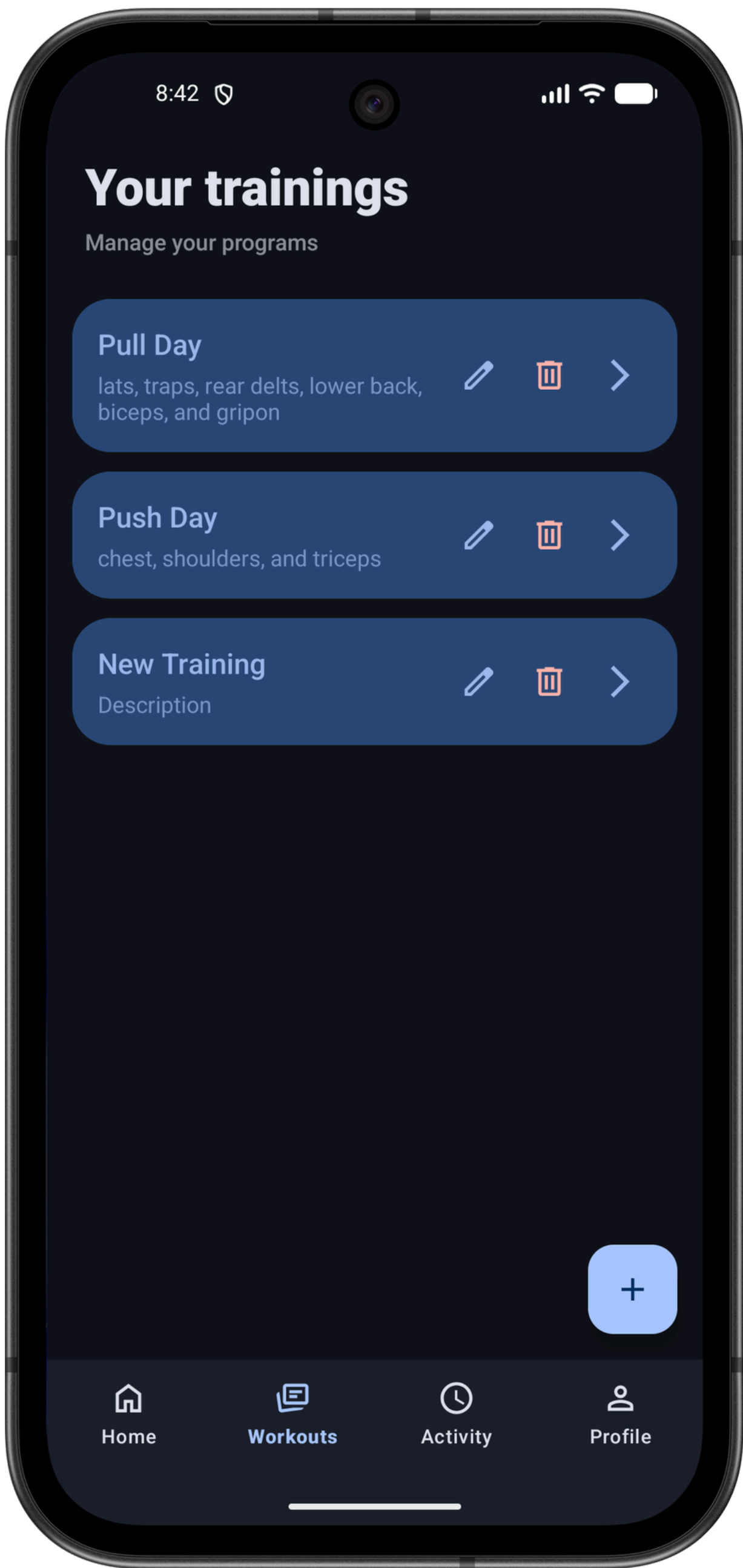
Profile

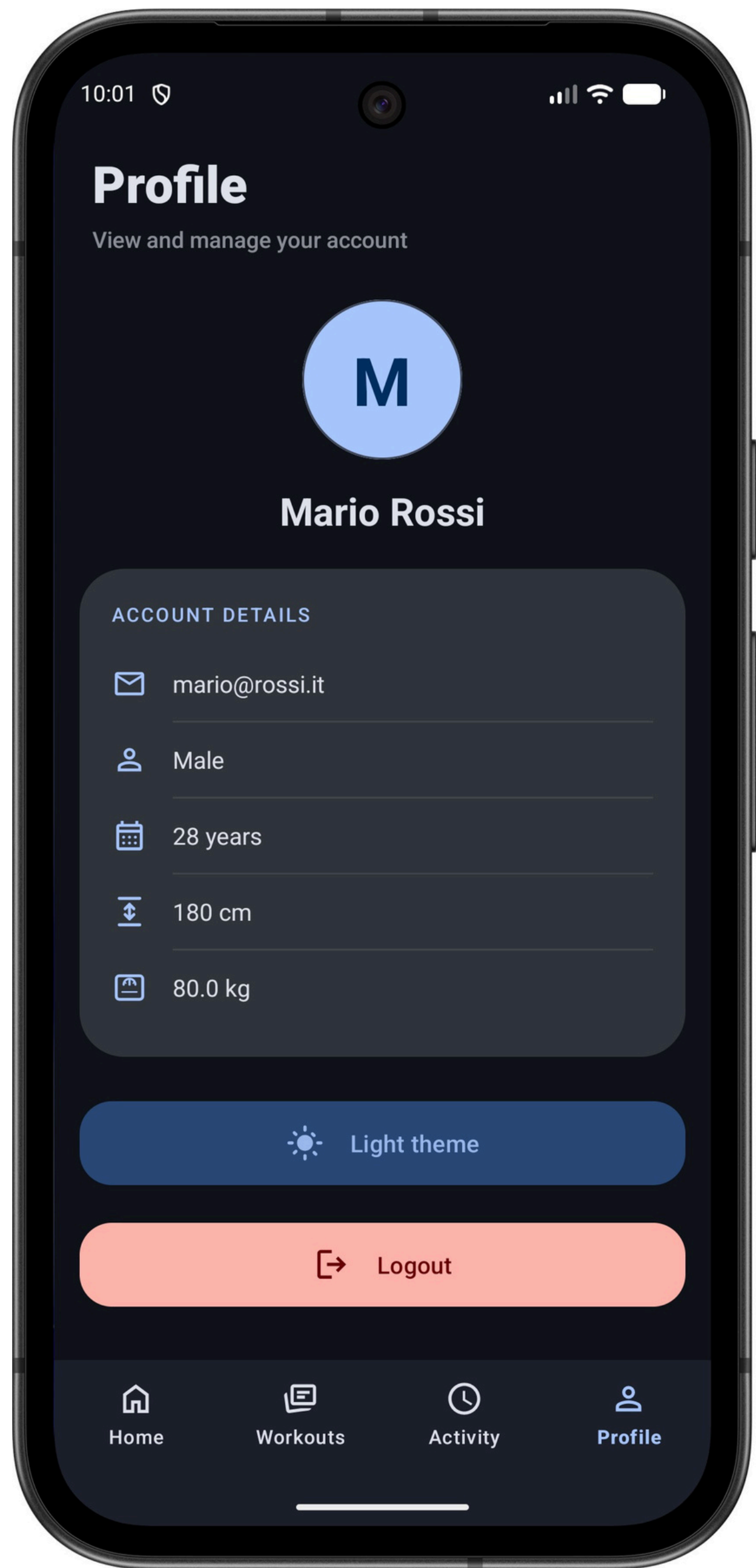
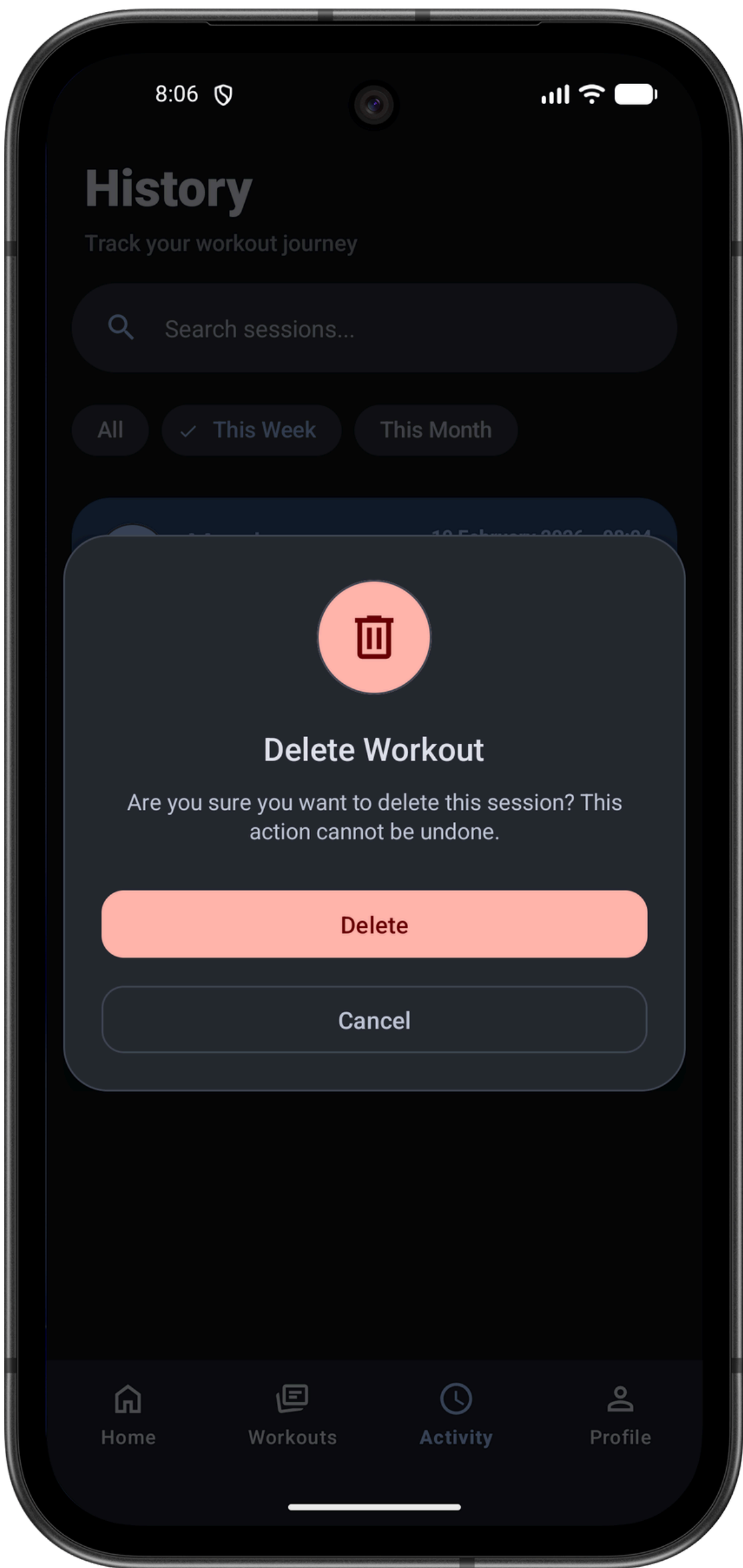
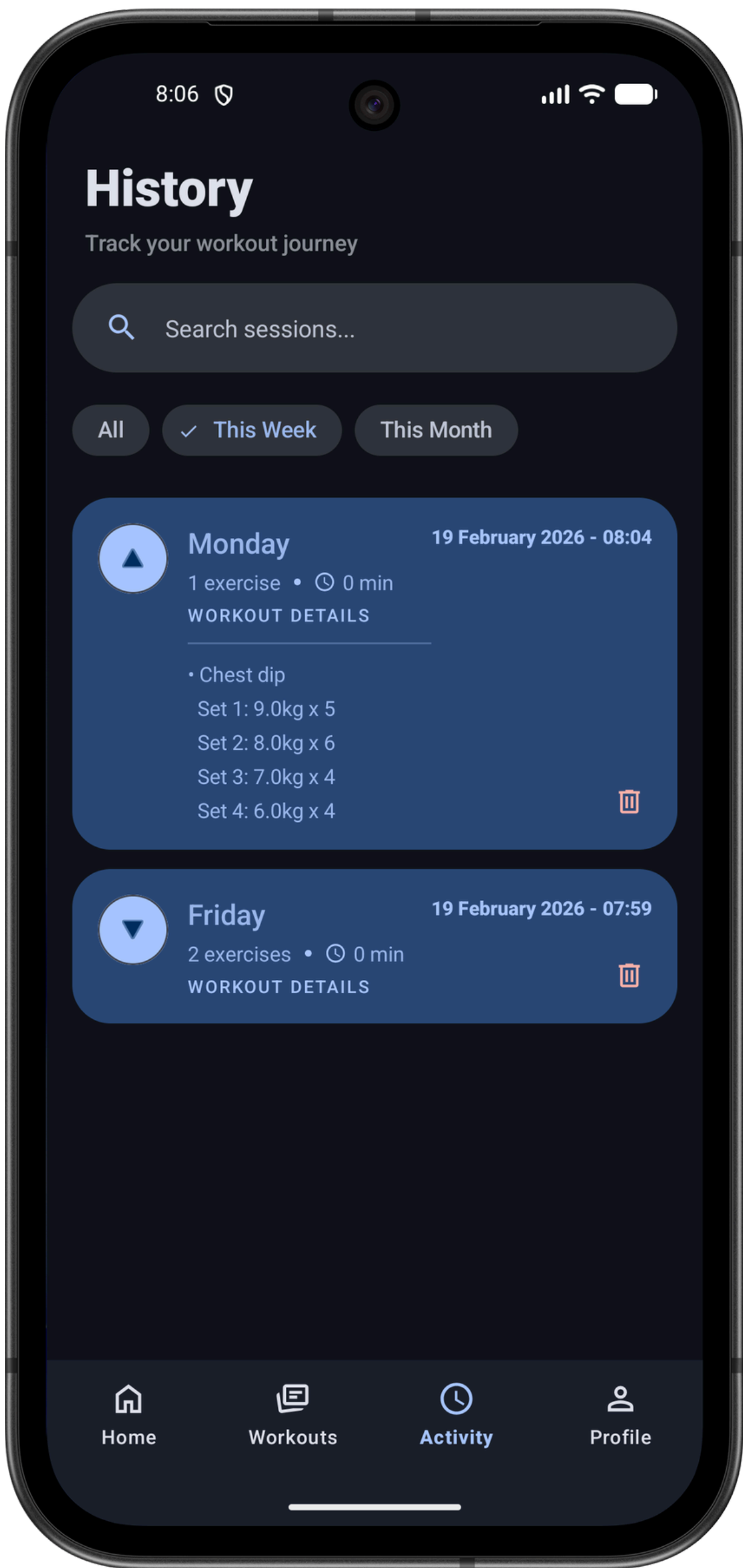
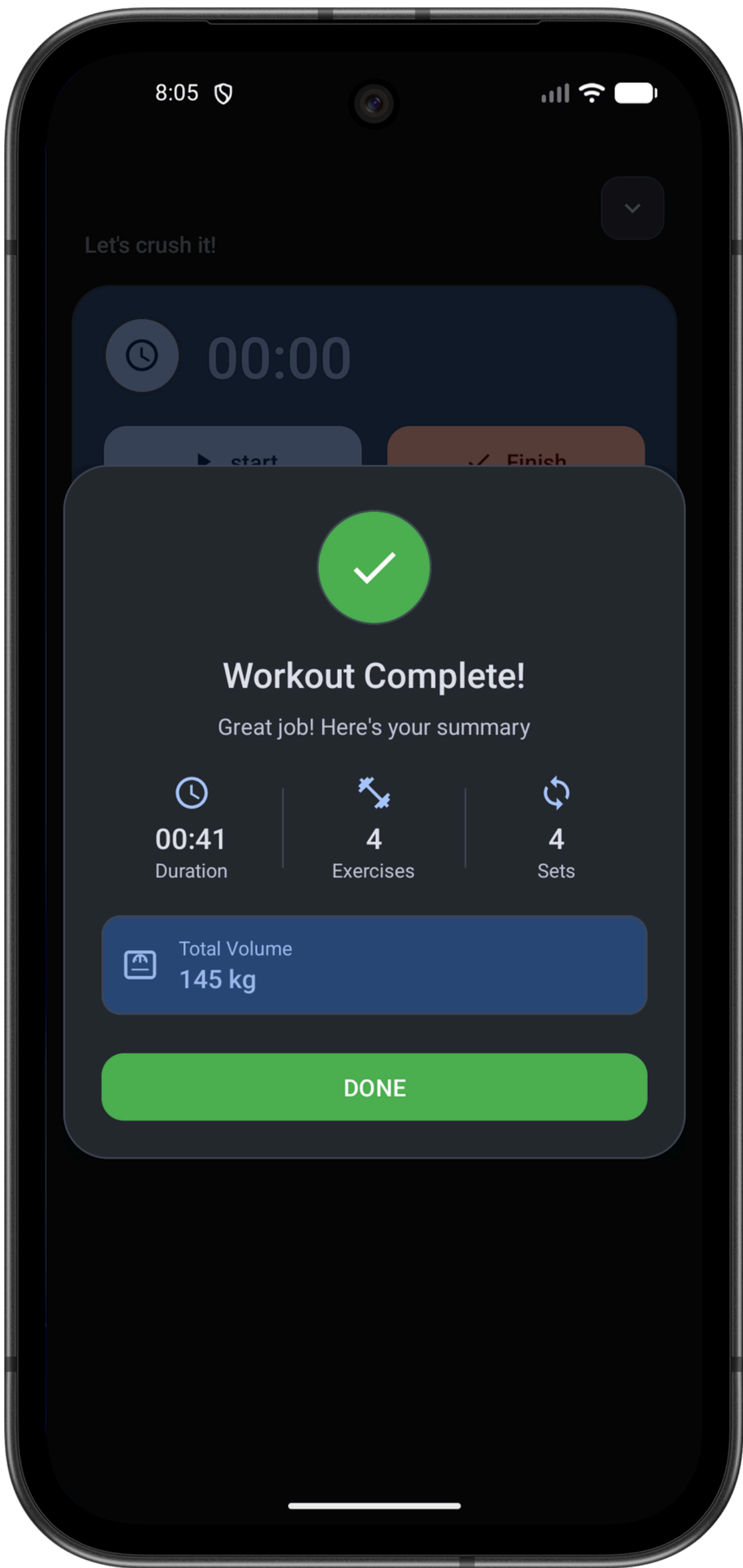
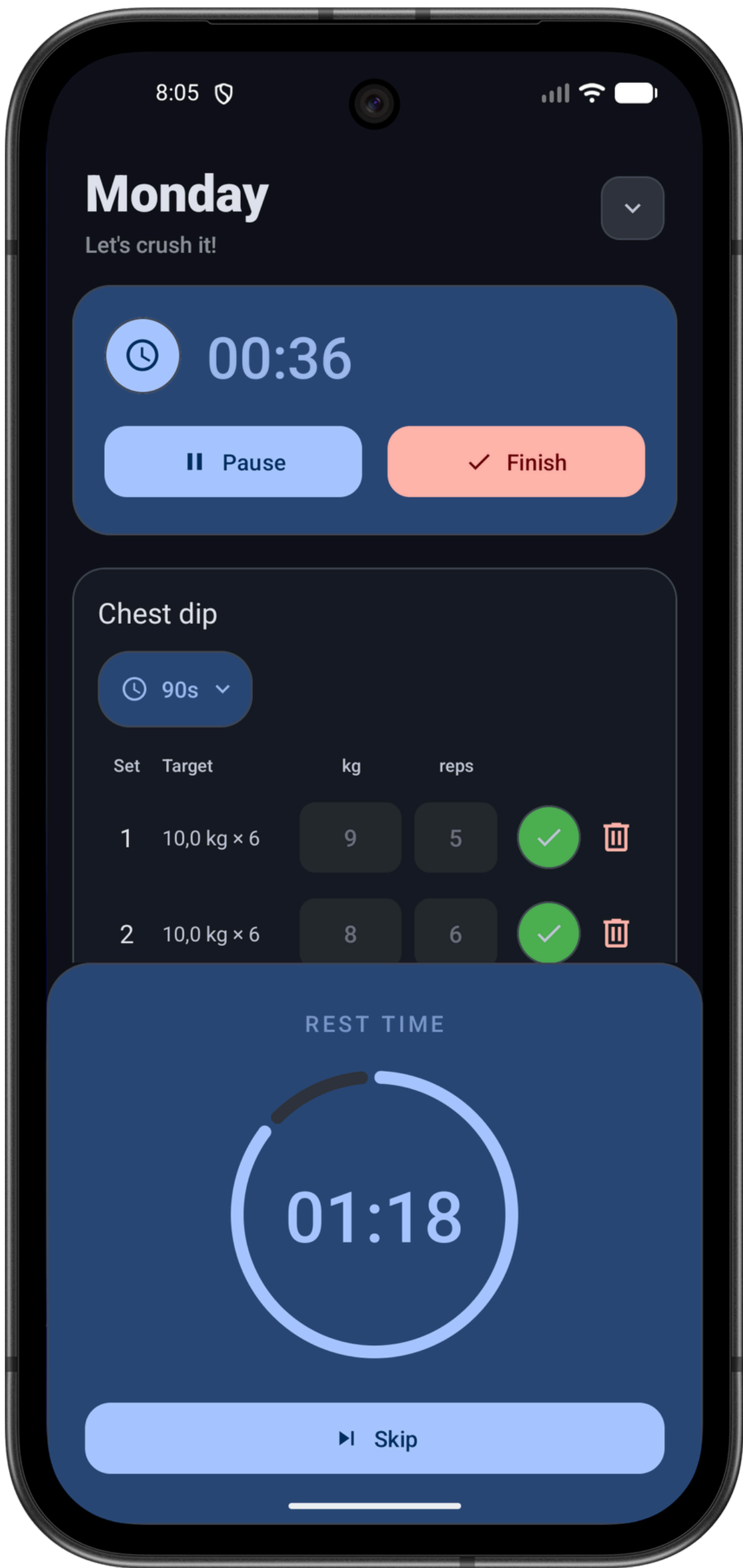


ProfileFragment

Schermate PushApp per il tema scuro







4. Conclusioni

Il progetto PushApp rappresenta una soluzione completa per il monitoraggio dell'allenamento di resistenza, evolvendo il concetto di semplice "tracker" in uno strumento analitico avanzato. Attraverso un'architettura MVVM rigorosa e modulare, l'applicazione integra con successo la persistenza locale robusta (Room) con la flessibilità del cloud (Firebase), garantendo un'esperienza utente fluida e "Offline-First". La capacità di gestire strutture dati gerarchiche complesse e di visualizzare metriche di performance in tempo reale dimostra la maturità tecnica del sistema, ponendo solide basi per un ecosistema digitale dedicato al fitness.

4.1 Risultati Chiave e Punti di Forza

Tra i principali risultati tecnici e funzionali ottenuti, evidenziamo:

- **Architettura "Offline-First" Resiliente:** Grazie all'implementazione del Repository Pattern, l'app garantisce la totale operatività anche in assenza di rete, utilizzando il database locale LocalDatabase come Single Source of Truth e sincronizzando i dati su Firestore in background.
- **Modellazione Dati Complessa:** La gestione delle relazioni "uno-a-molti" nidificate (Training → Routine → Esercizio → Serie) è stata risolta efficacemente tramite l'uso avanzato delle annotazioni @Relation e @Embedded di Room, permettendo query performanti su strutture dati profonde.
- **Interfaccia Utente Reattiva:** L'adozione di Nested RecyclerViews all'interno di WorkoutFragment e EditRoutineFragment ha permesso di rappresentare visivamente la gerarchia degli esercizi e delle serie senza compromettere la fluidità dello scorrimento o la gestione degli eventi di input.
- **Integrazione Ecosistema Esterno:** L'orchestrazione tra servizi eterogenei — autenticazione Firebase, database Firestore e Ninja API per il catalogo esercizi — è gestita in modo trasparente per l'utente finale, arricchendo l'app con contenuti dinamici e aggiornati.

4.2 Sfide Incontrate e Lezioni Apprese

Lo sviluppo di PushApp ha presentato sfide tecniche significative, superate attraverso soluzioni mirate:

- **Gestione della Concorrenza in Java:** L'assenza delle Coroutines di Kotlin ha richiesto un'attenta gestione dei thread. Abbiamo implementato un `ExecutorService` dedicato per le operazioni di scrittura su database e definito interfacce di callback (`FirestoreCallback`, `TrainingCallback`) per gestire le risposte asincrone senza bloccare il Main Thread.
- **Sincronizzazione Bidirezionale Manuale:** Mantenere la coerenza tra il database relazionale locale (Room) e il database a documenti (Firestore) ha richiesto una logica di mappatura rigorosa nei Data Source. In particolare, la gestione degli ID univoci (UUID) e la propagazione delle modifiche (es. eliminazione di una routine) hanno necessitato di una gestione transazionale accurata.
- **Persistenza dello Stato di Allenamento:** Garantire che il timer e i dati della sessione attiva non andassero persi durante la navigazione o la chiusura dell'app ha richiesto l'implementazione di un `SessionManager` basato su `SharedPreferences` e una logica di ripristino stato nel `WorkoutViewModel`.

4.3 Sviluppi Futuri e Prospettive

La roadmap di sviluppo di PushApp prevede l'implementazione di funzionalità avanzate per elevare ulteriormente l'esperienza utente:

- **Algoritmi di Raccomandazione del Carico:** Implementare una logica predittiva nel `TrainingViewModel` che, analizzando lo storico dei massimali e del volume nel `HistoryRepository`, suggerisca automaticamente il peso da utilizzare per garantire il sovraccarico progressivo.
- **Companion App per Wear OS:** Sviluppare un'estensione per smartwatch che permetta il logging rapido delle serie e il controllo del timer direttamente dal polso, riducendo la dipendenza dallo smartphone durante l'allenamento.
- **Modulo Social & Community:** Introdurre la possibilità di condividere i propri Training e i risultati delle sessioni con altri utenti, creando un feed sociale integrato che favorisca la competizione sana e la motivazione.
- **Modulo Food:** Integrare un'interfaccia per la gestione del proprio piano alimentare e per il tracciamento dei macronutrienti.
- **Personalizzazione esercizi:** Aggiungere la possibilità di creare esercizi personalizzati.

In conclusione, PushApp non è solo un diario digitale, ma una piattaforma scalabile pronta per evolversi insieme alle esigenze degli atleti più esigenti.